

计算机体系结构

L01 简单流水线

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ②

多处理器 (线程级并行) ⑤

加速器 (数据并行) ③

简单流水线

单周期 CPU

简单5级流水线 📝

指令相关性 (RAW)

Stall 逻辑

旁路 (Bypassing)

流水线深度

复杂流水线 ②

分支预测 ③

指令并行高级技术 ②

处理器性能 (回顾)

单指令周期数 (CPI)

□ CPI: 平均情况下执行一条指令需要的周期数

✓ $IPC = 1/CPI$

- 比CPI更常用（因为IPC越高越好）

✓ 不同指令需要的周期数不同

- E.g., “add” 通常需要1个周期, “divide” 需要10多个周期

✓ 程序的CPI取决于该程序不同指令的占比

程序执行时间 = 指令数 * 单指令执行周期数 * 单周期时间

□ 程序执行的指令数 (Instruction Count)

- 由程序本身, 编译器, 指令集 (ISA) 决定

□ 单指令执行周期数 (Cycles Per Instruction, CPI)

- 执行一条指令平均需要多少周期
- 由程序本身, 编译器, 指令集 (ISA), 体系结构决定

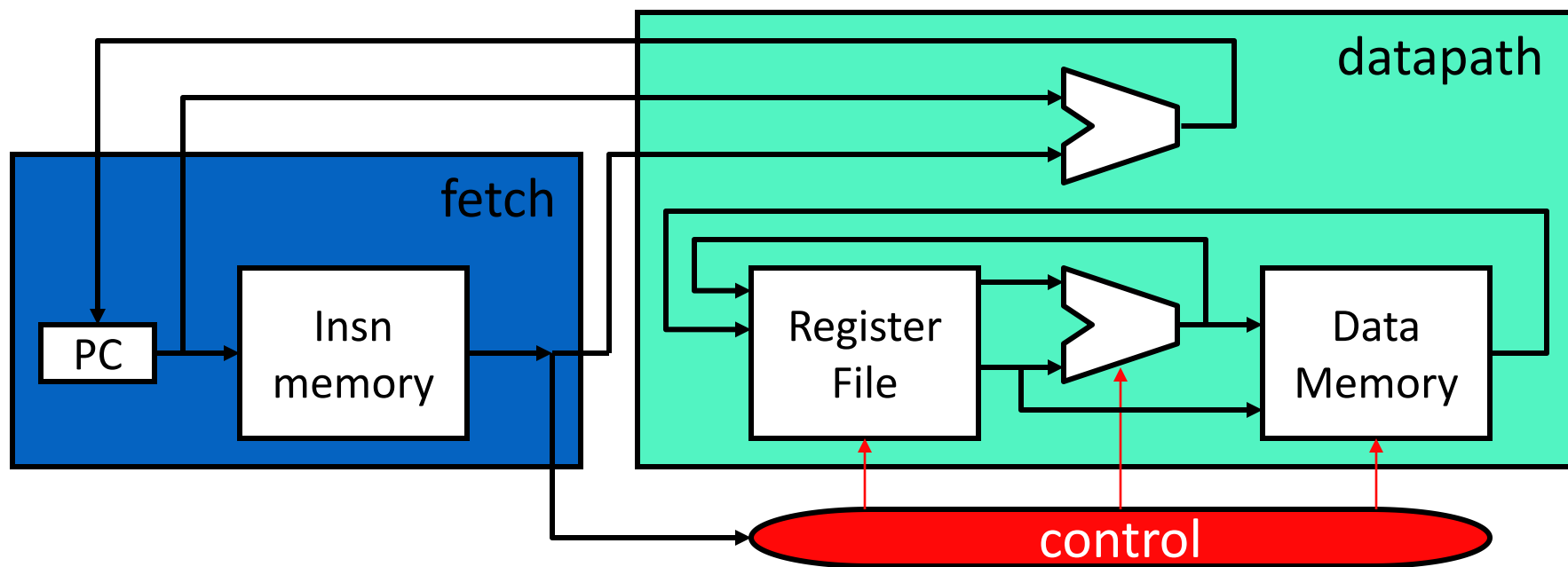
□ 单周期时间 (Clock Period)

- 其倒数是时钟频率: 0.5 Ghz to 4 Ghz
- 由体系结构, 制造工艺等因素决定

为了最小化程序执行时间, 需要最小化上面每一项, 但通常是此消彼长、相互冲突!

单周期CPU (回顾)

设计一个ISA



- **数据通路(data path):** 完成计算 (registers, ALUs, etc.)
 - ✓ ISA 特定: 可以实现每条指令
- **控制通路(control path):** 决定执行哪种运算
 - ✓ 将数据通过数据通路进行路由 (哪些寄存器, 哪个运算单元)
- **取指令:** 读取指令, 并进行译码
- **Fetch** → **Decode** → **Execute** “循环”

单周期CPU数据通路 (MIPS ISA)

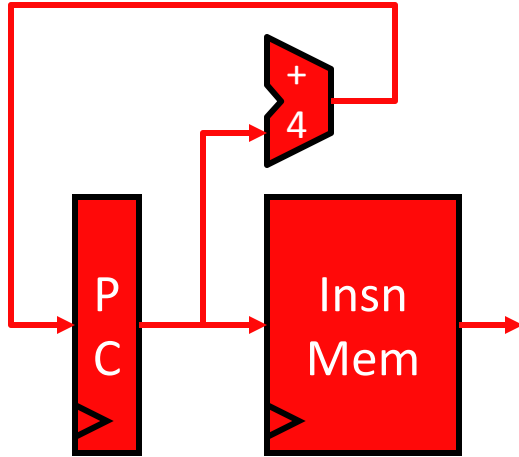
□ MIPS: 32-bit 指令, 32个寄存器 \$0, \$2... \$31

□ 仅考虑以下指令

<code>add \$1,\$2,\$3</code>	<code>\$1 = \$2 + \$3</code>	(add)
<code>addi \$1,\$2,3</code>	<code>\$1 = \$2 + 3</code>	(add immed)
<code>lw \$1,4(\$3)</code>	<code>\$1 = Memory[4+\$3]</code>	(load)
<code>sw \$1,4(\$3)</code>	<code>Memory[4+\$3] = \$1</code>	(store)
<code>beq \$1,\$2,PC_relative_target</code>		(branch equal)
<code>j absolute_target</code>		(unconditional jump)

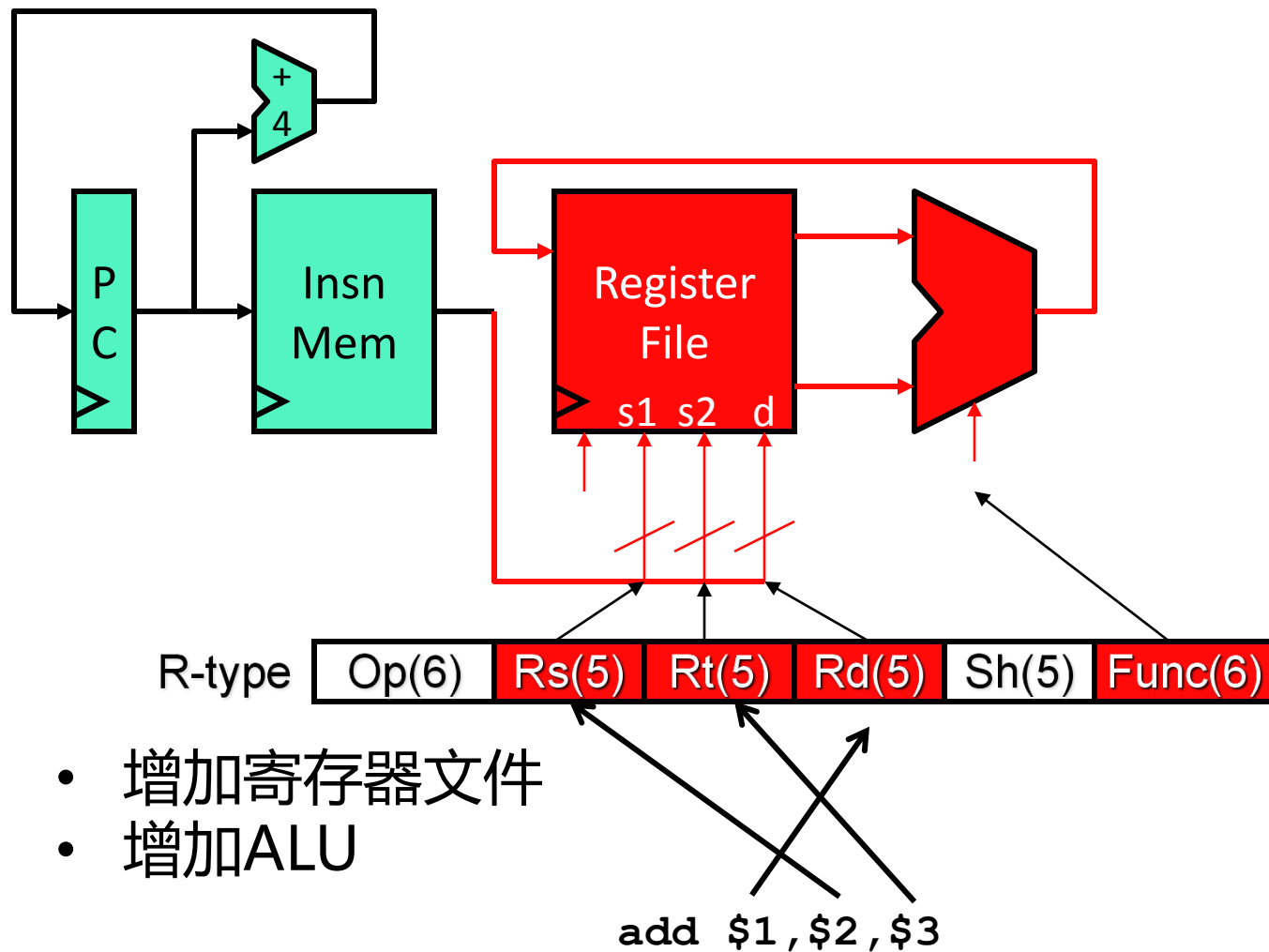
*其他指令和上述指令类似

Fetch

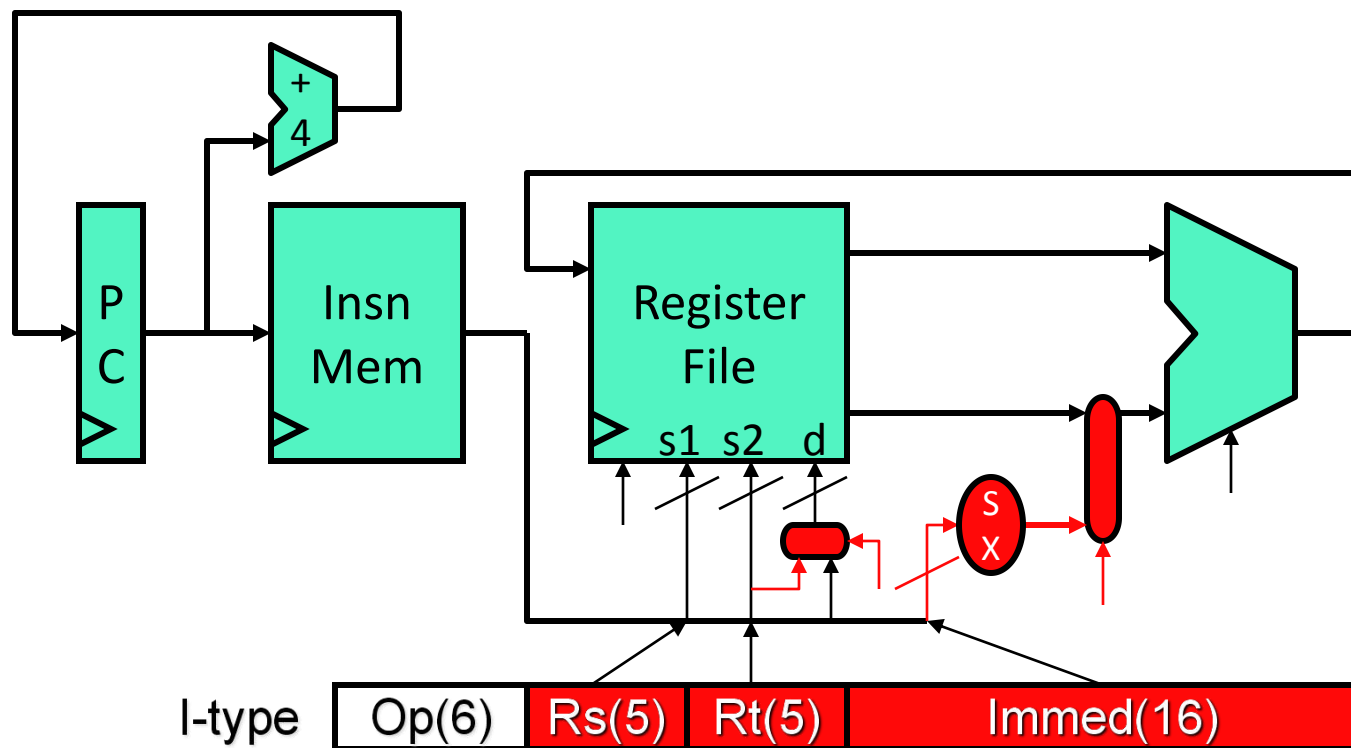


- PC和指令内存(假设指令内存和数据内存分离)
- $PC + 4$ 运算单元计算下一条指令PC

第一条指令: add



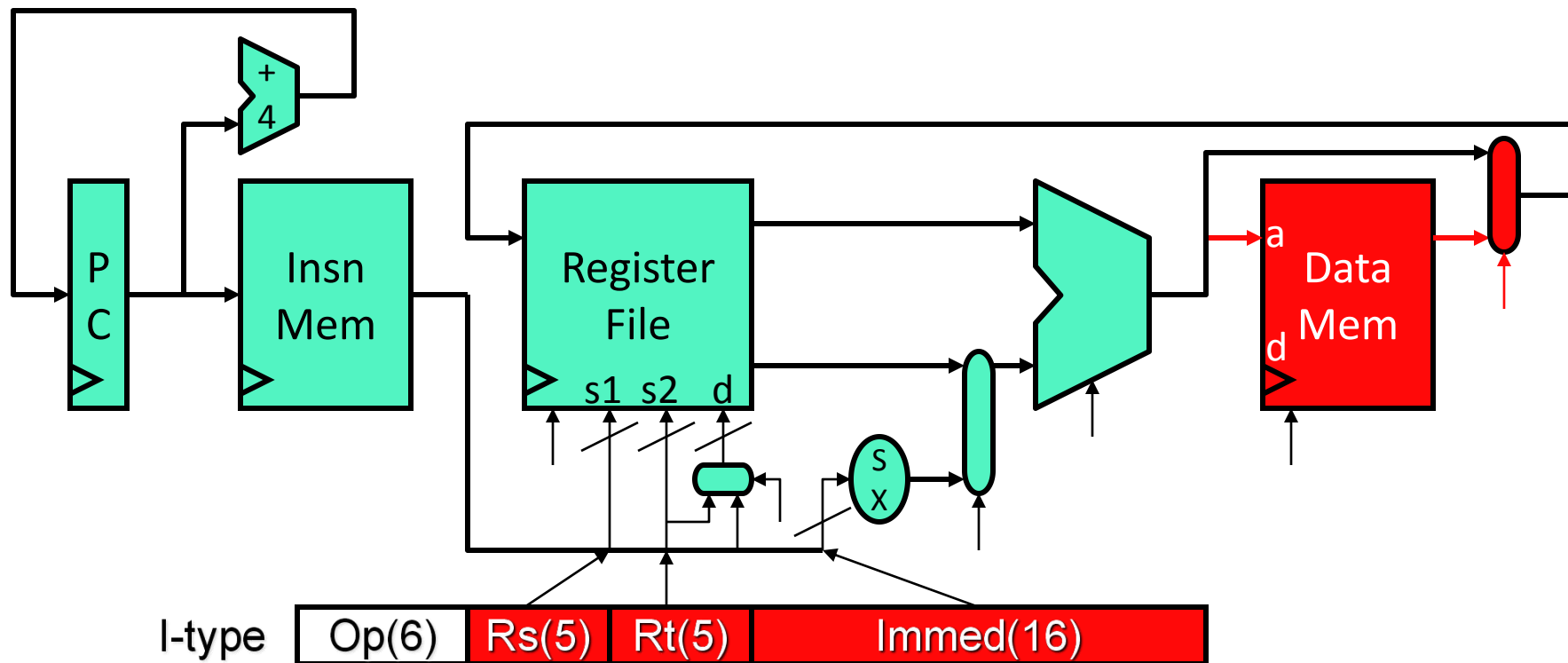
第二条指令: addi



- 目标寄存器可能是Rd或Rt
- 为ALU的第二个操作数增加符号扩展和多路选择单元

`addi $1, $2, 8`

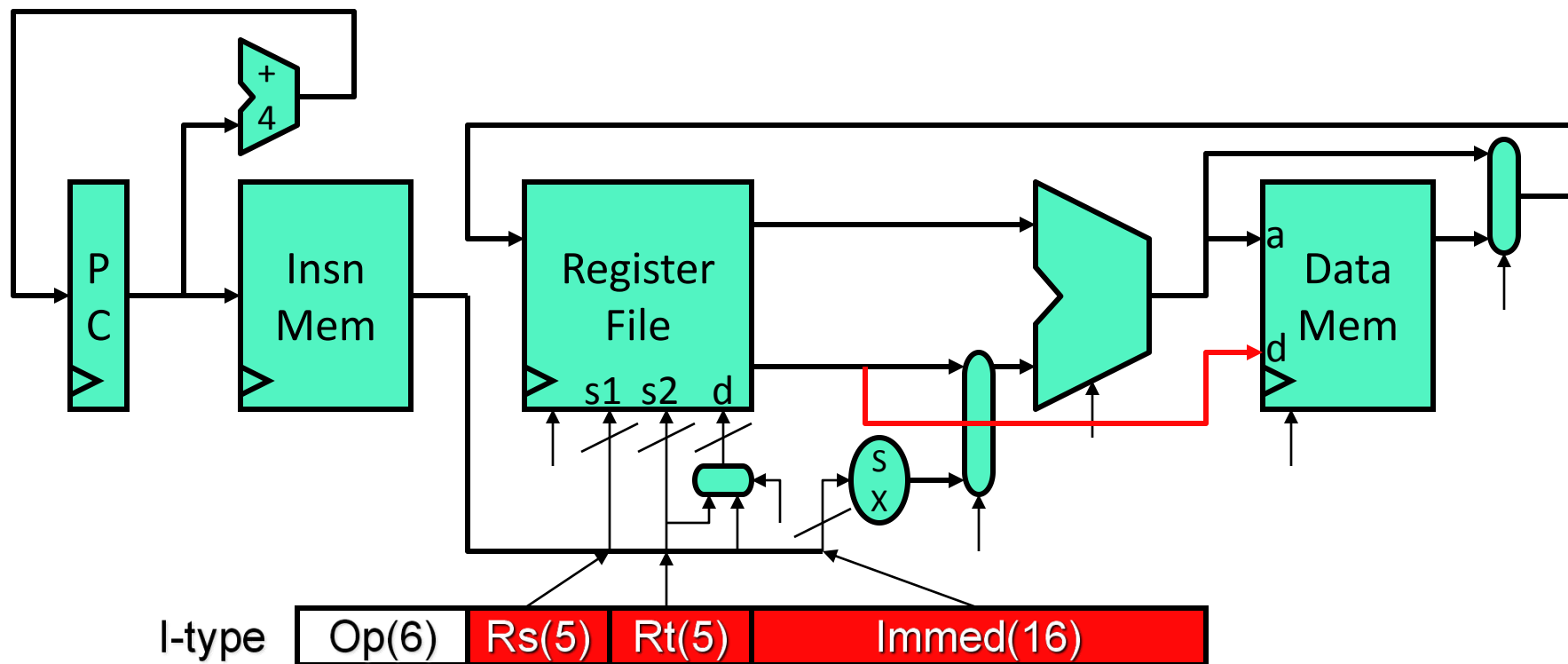
第三条指令: lw



- 增加数据内存，地址为ALU的输出
- 寄存器写端口增加二选一：内存输出或ALU输出

`lw $1, 8($2)`

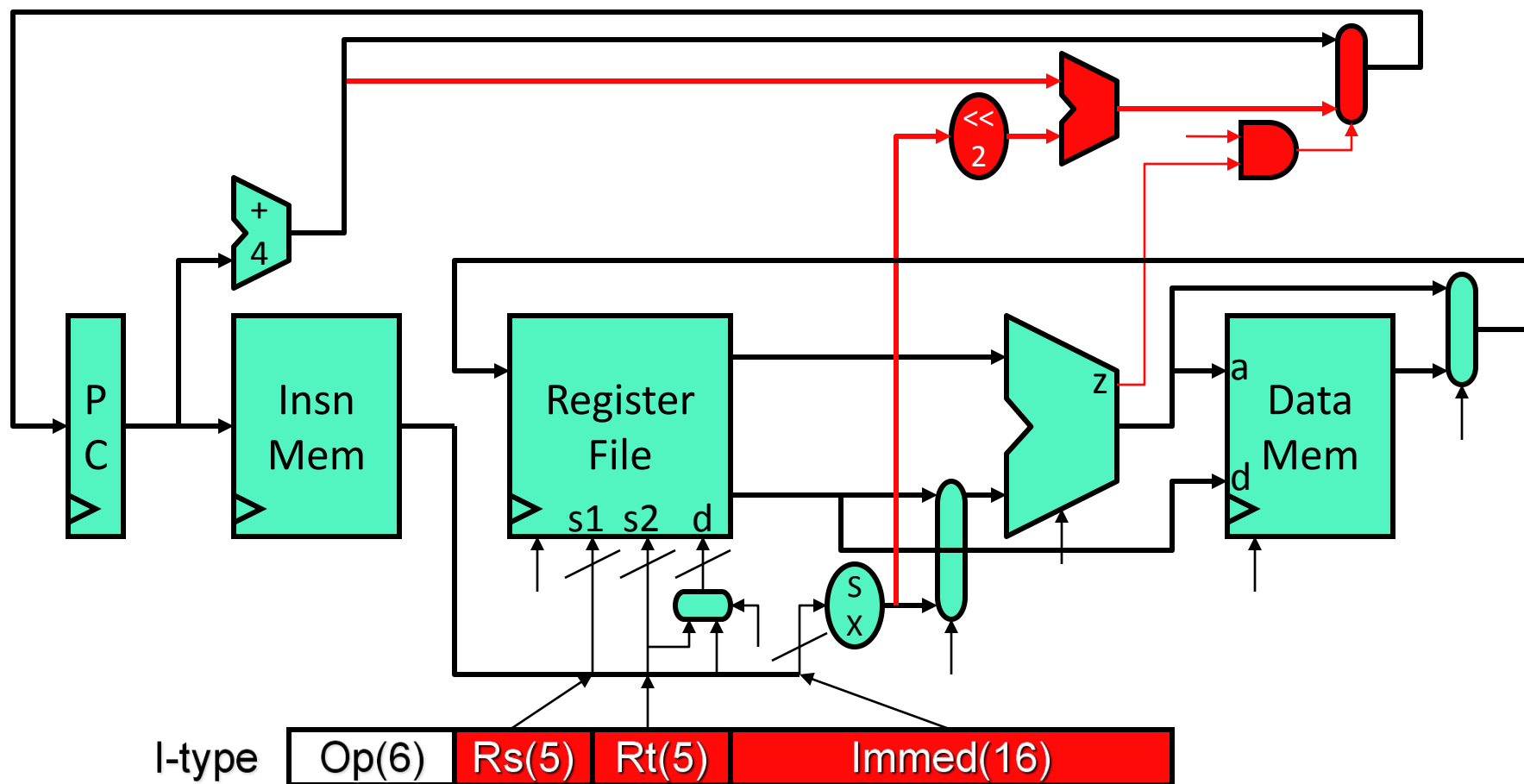
第四条指令: sw



- 增加从寄存器第二个输出端口到数据内存输入端口的通路

sw \$1, 8(\$2)

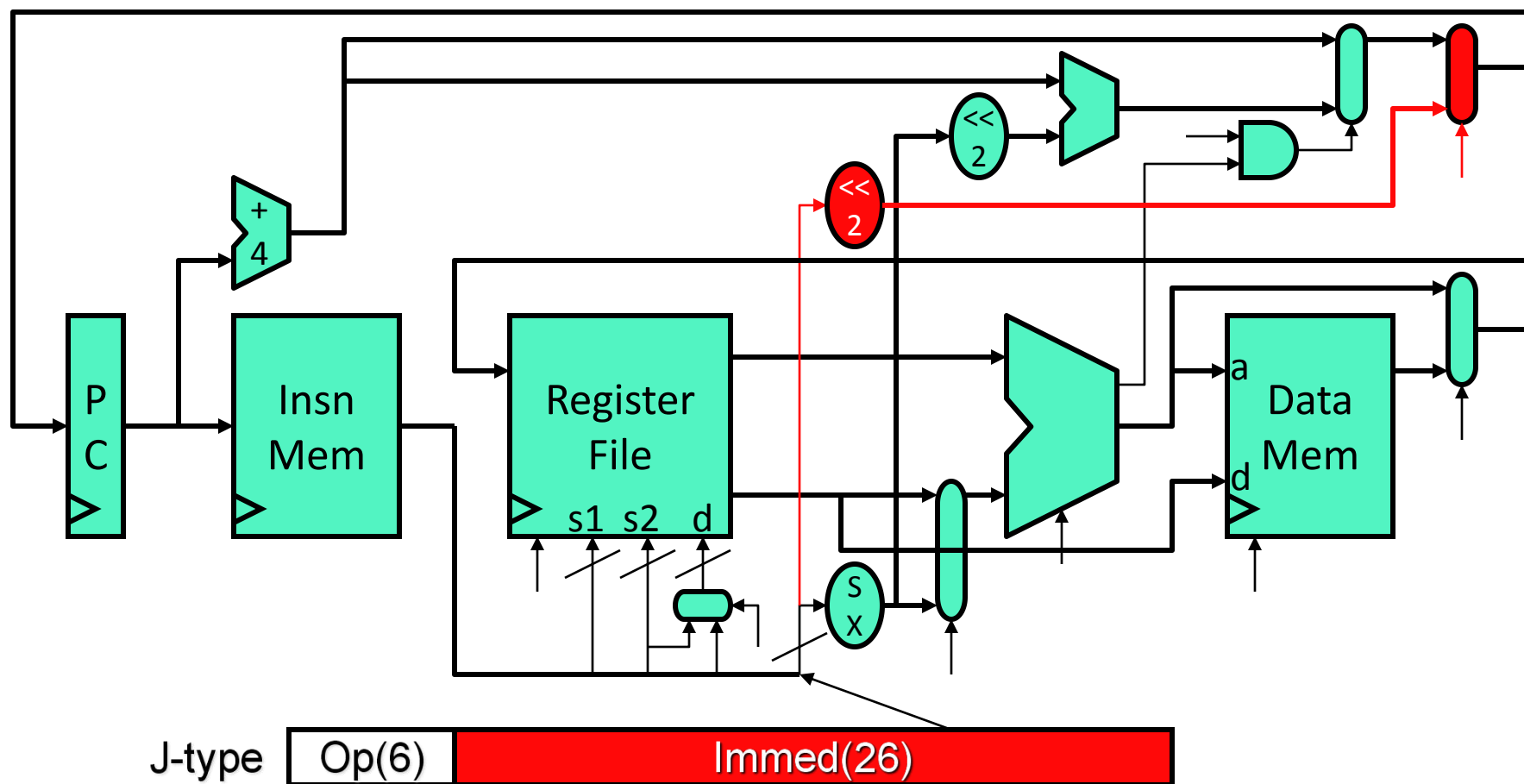
第五条指令: beq



- 增加左移和加法器计算PC-relative branch target
- 增加PC输入端的二选一：PC+4或分支目标

`beq $1, $2, 8`

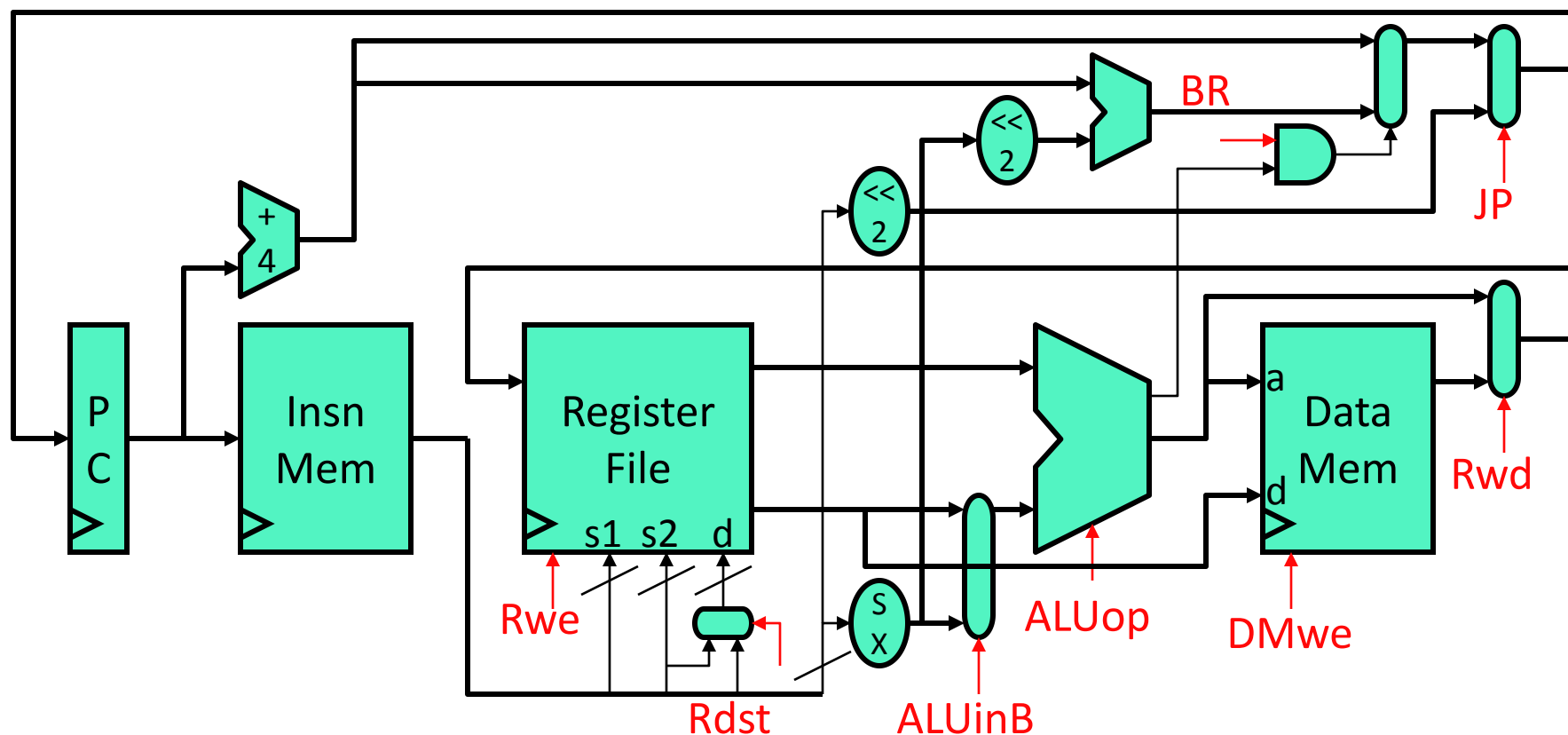
第六条指令: j



- 增加移位器计算26-bit立即数左移
- PC输入端再增加一路选择：跳转地址

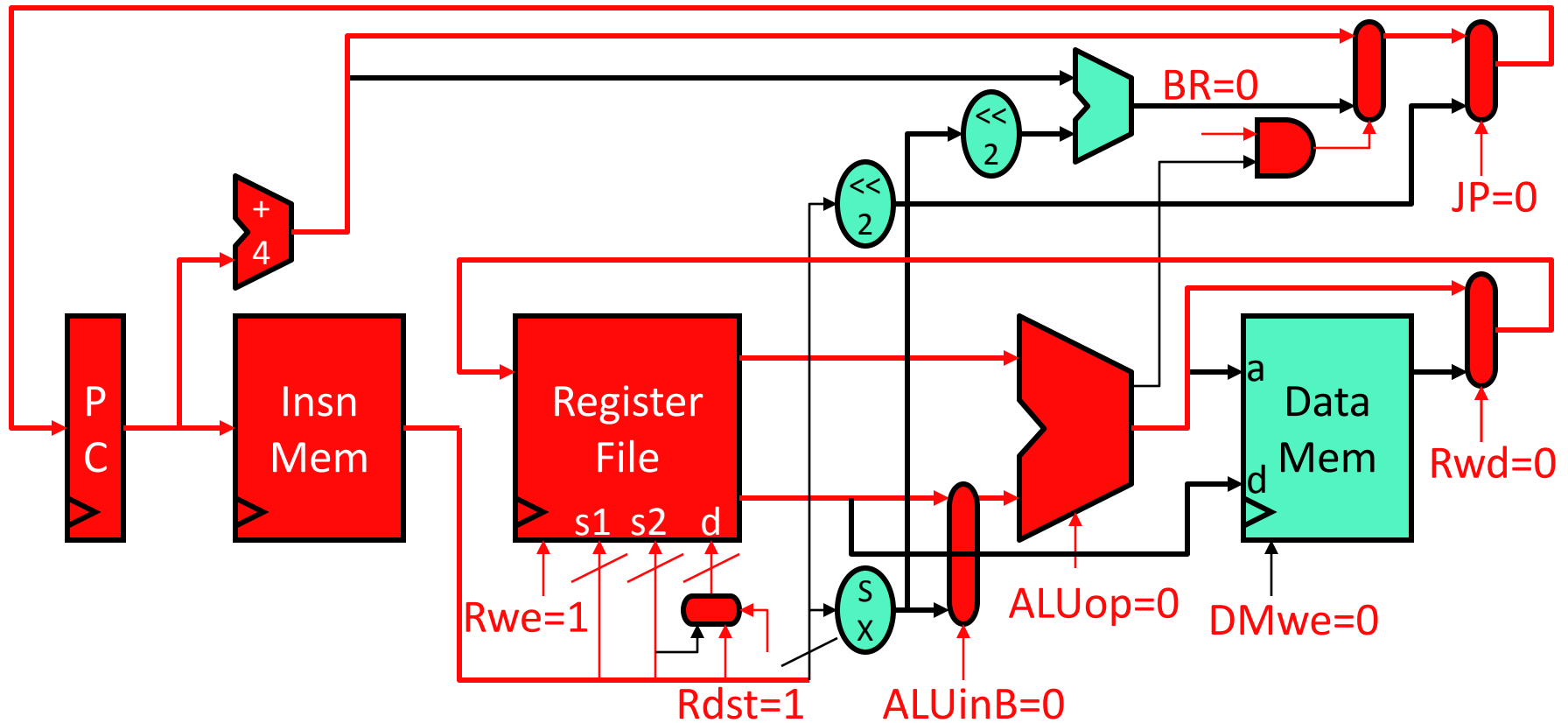
j 100

控制信号



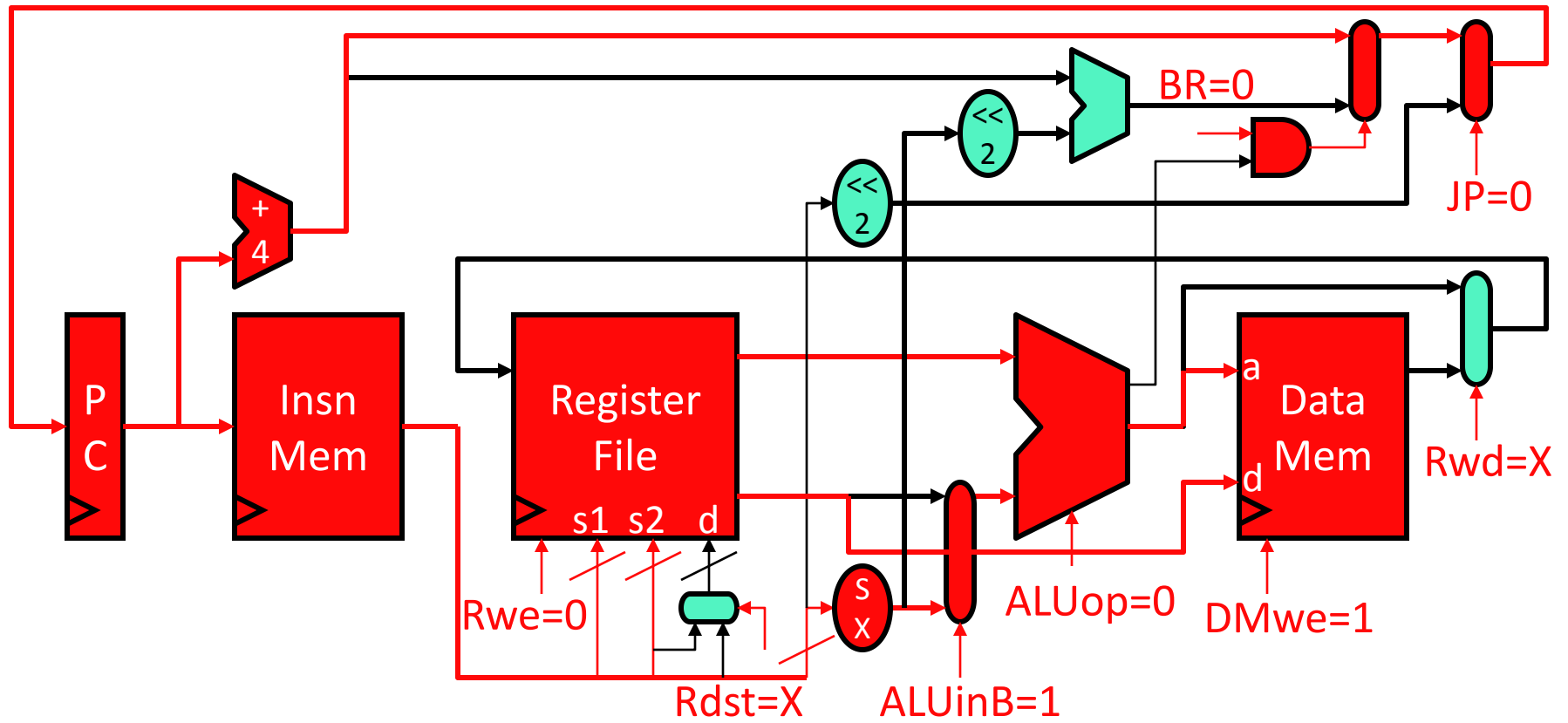
- 共有8个控制信号
 - ✓ 多路选择信号，寄存器/内存写端口使能信号等
 - ✓ 真实的数据通路通常由300-500个控制信号

add指令控制信号



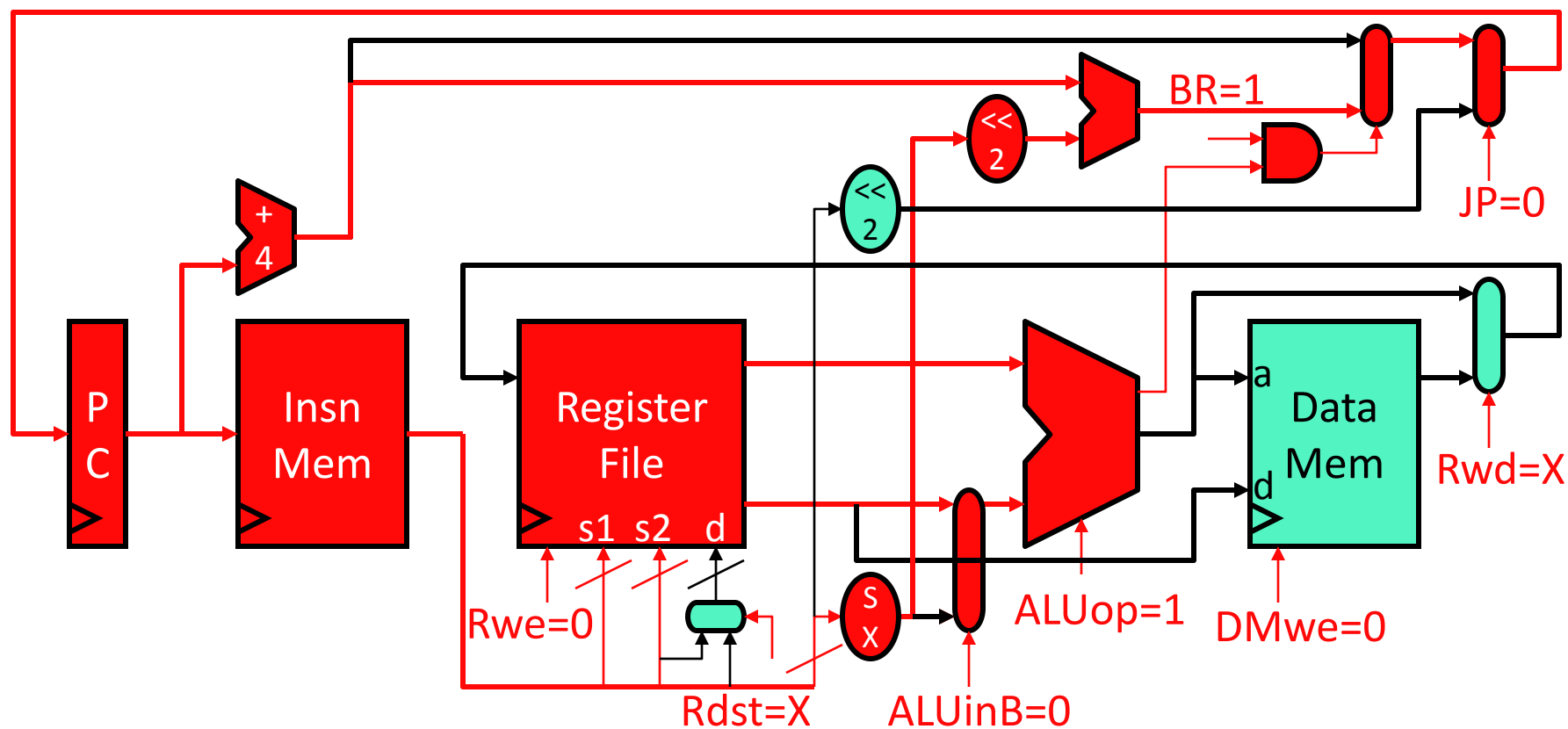
`add $4, $3, $4`

sw指令控制信号



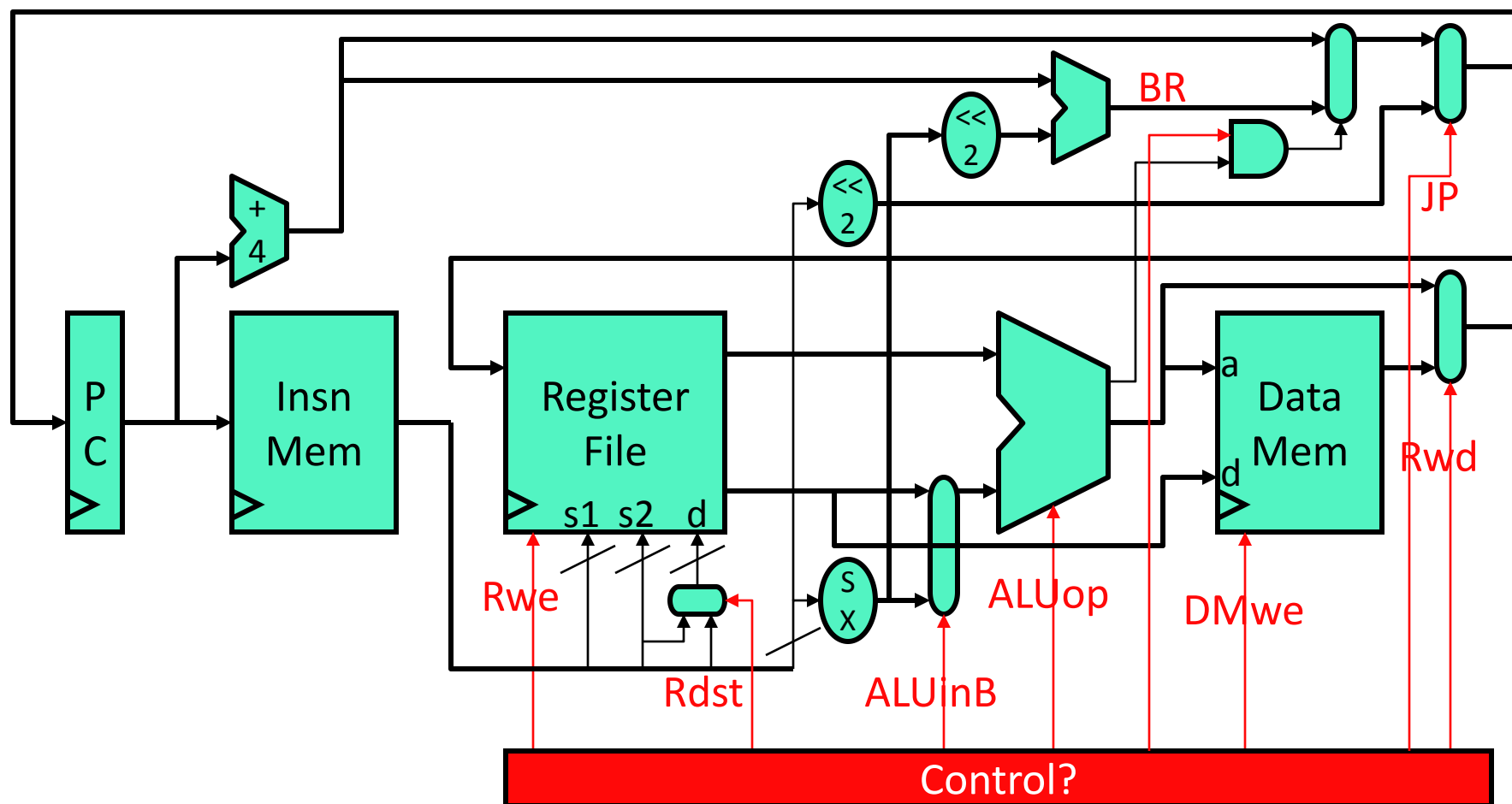
`sw $4, 0($2)`

beq指令控制信号



```
beq $5, $6, array_sum_loop
```

控制逻辑如何实现

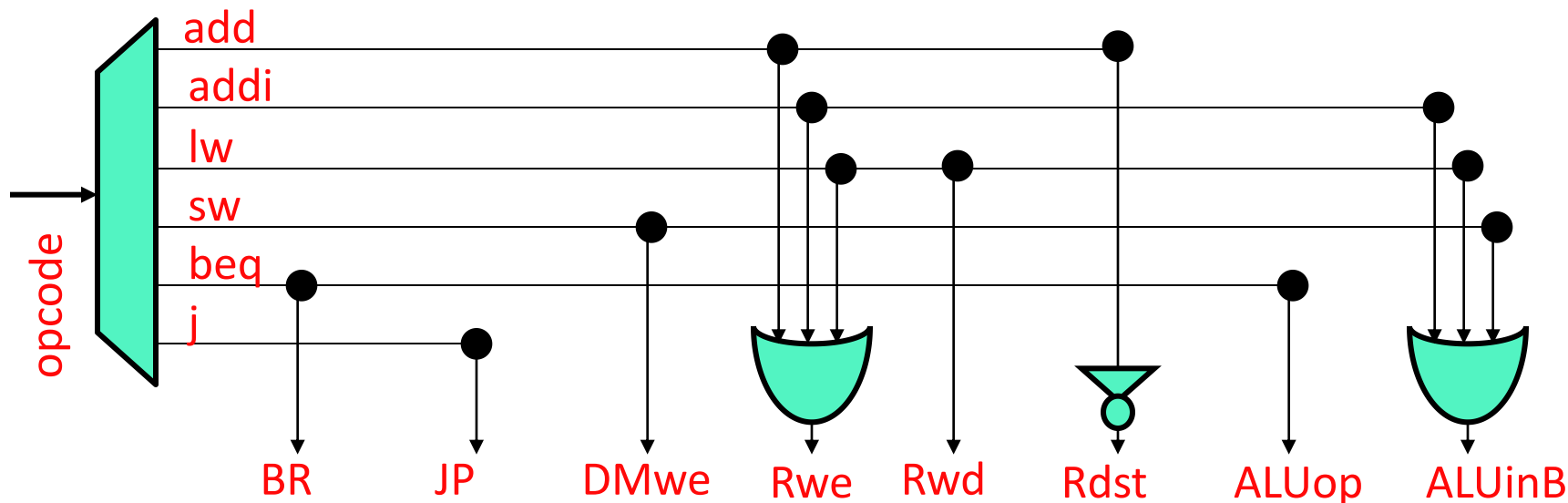


控制逻辑如何实现

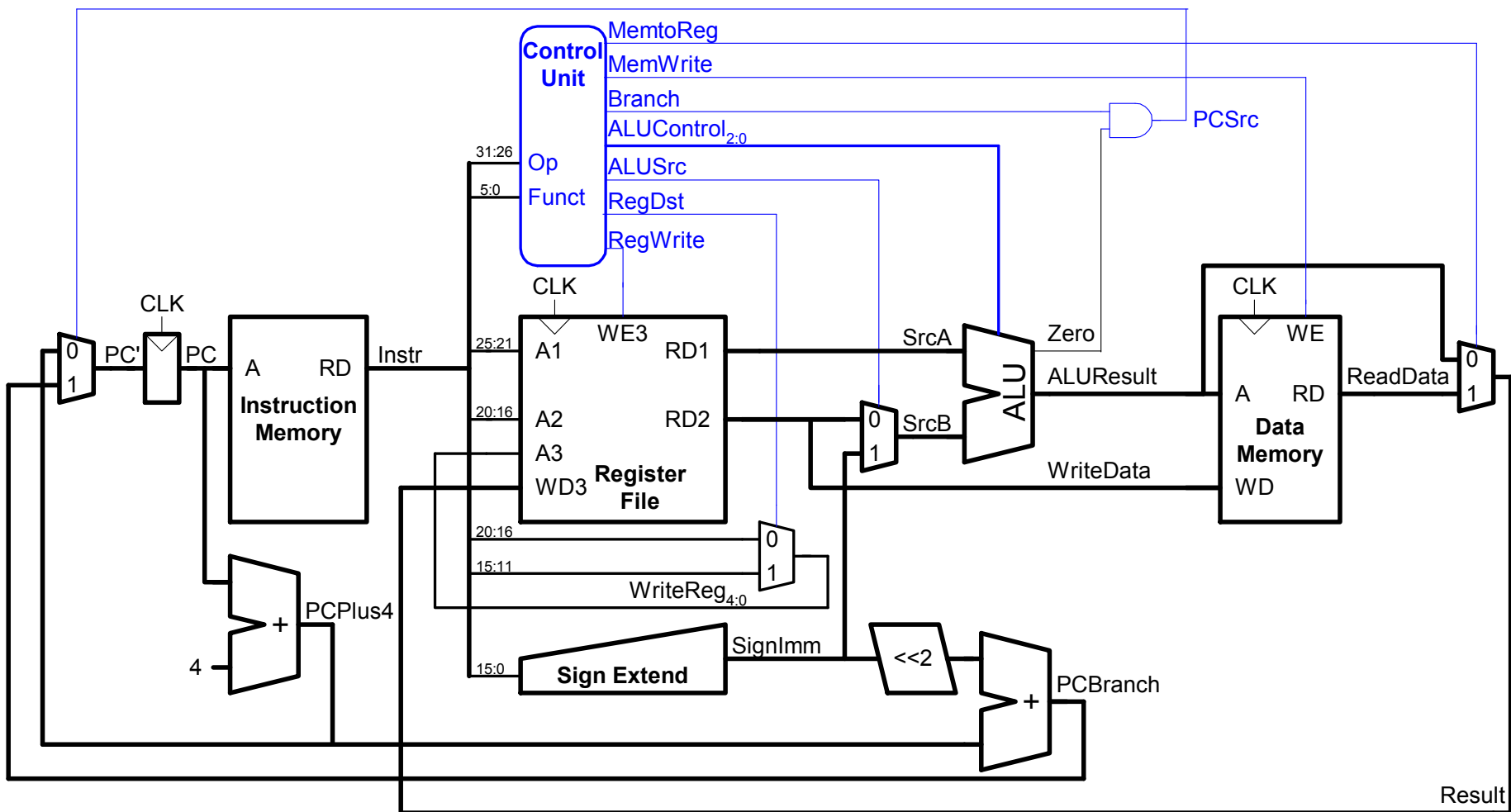
□ 真实CPU有100+指令，300+控制信号

□ 采用逻辑电路

✓ 观察：很多信号有大量0或者大量1



单周期CPU



单周期CPU分析

□ 每条指令只需要一个周期完成

✓ $CPI = 1$

□ 每条指令的执行时间取决于最慢指令的执行时间

✓ 尽管很多指令不需要那么长时间

□ 系统时钟周期取决于完成**最慢指令**所需的时间

✓ 设计的关键路径由最慢指令的处理时间决定

简单5级流水线 (回顾)

流水线 (pipeline)

流水线

insn0.fetch	insn0.dec	insn0.exec	
	insn1.fetch	insn1.dec	insn1.exec

□ 核心思想

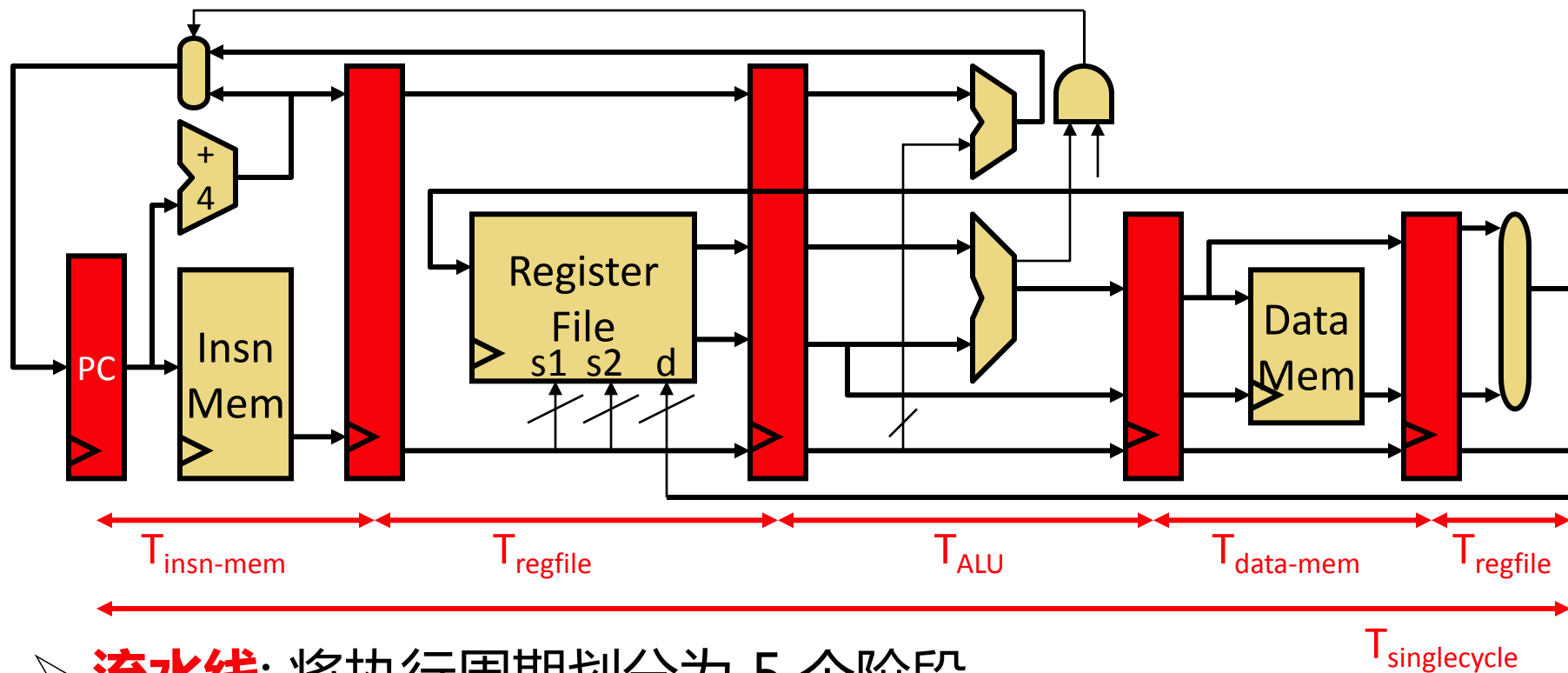
- 提升指令吞吐量，不是单个指令执行时间

□ 将指令执行划分成多个阶段

- 当指令从第1阶段进入到第2阶段, 下一条指令进入第1阶段
- 每条指令顺序经过所有阶段
- 所有指令经过的阶段数相同
- 并行方式: “指令-阶段 并行”

+ 相比单周期CPU，指令进入/离开的频率更快

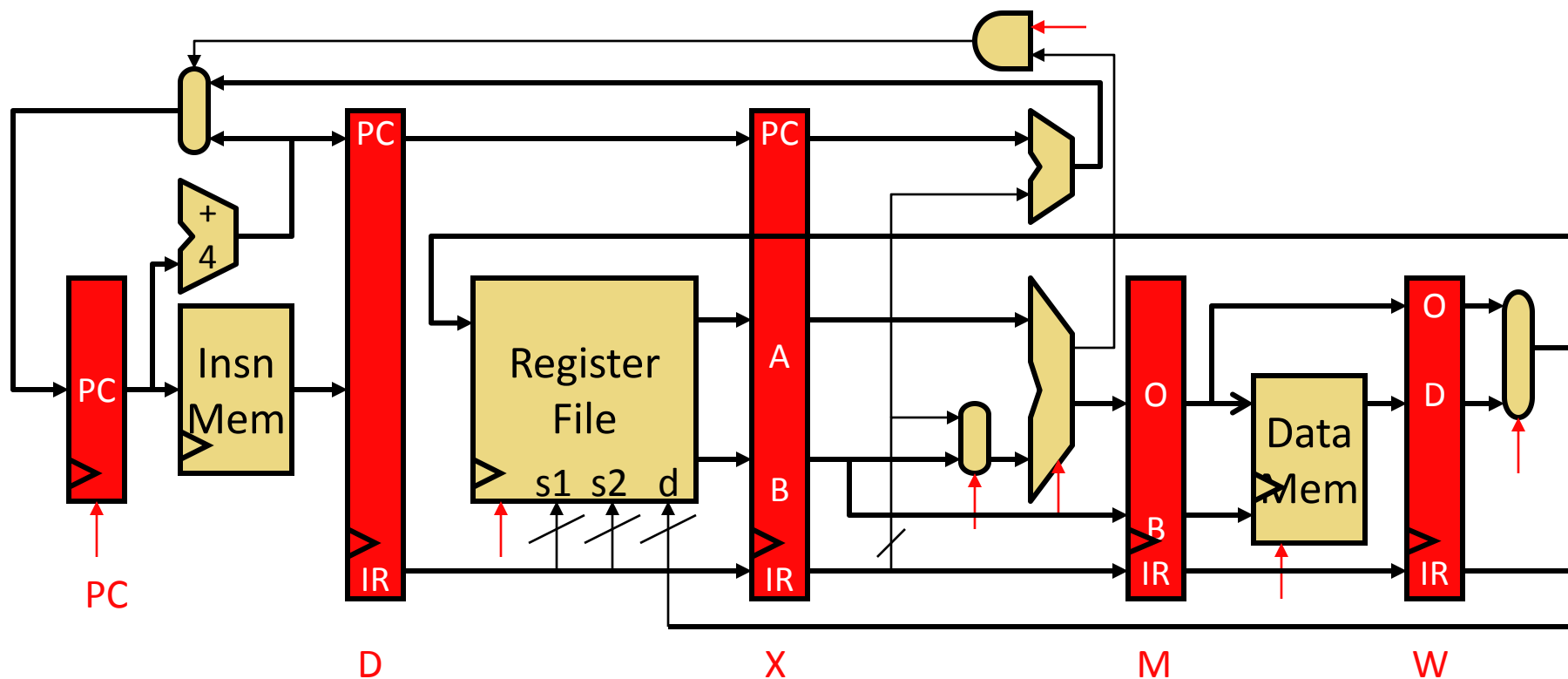
5级流水线



➤ 流水线: 将执行周期划分为 5 个阶段

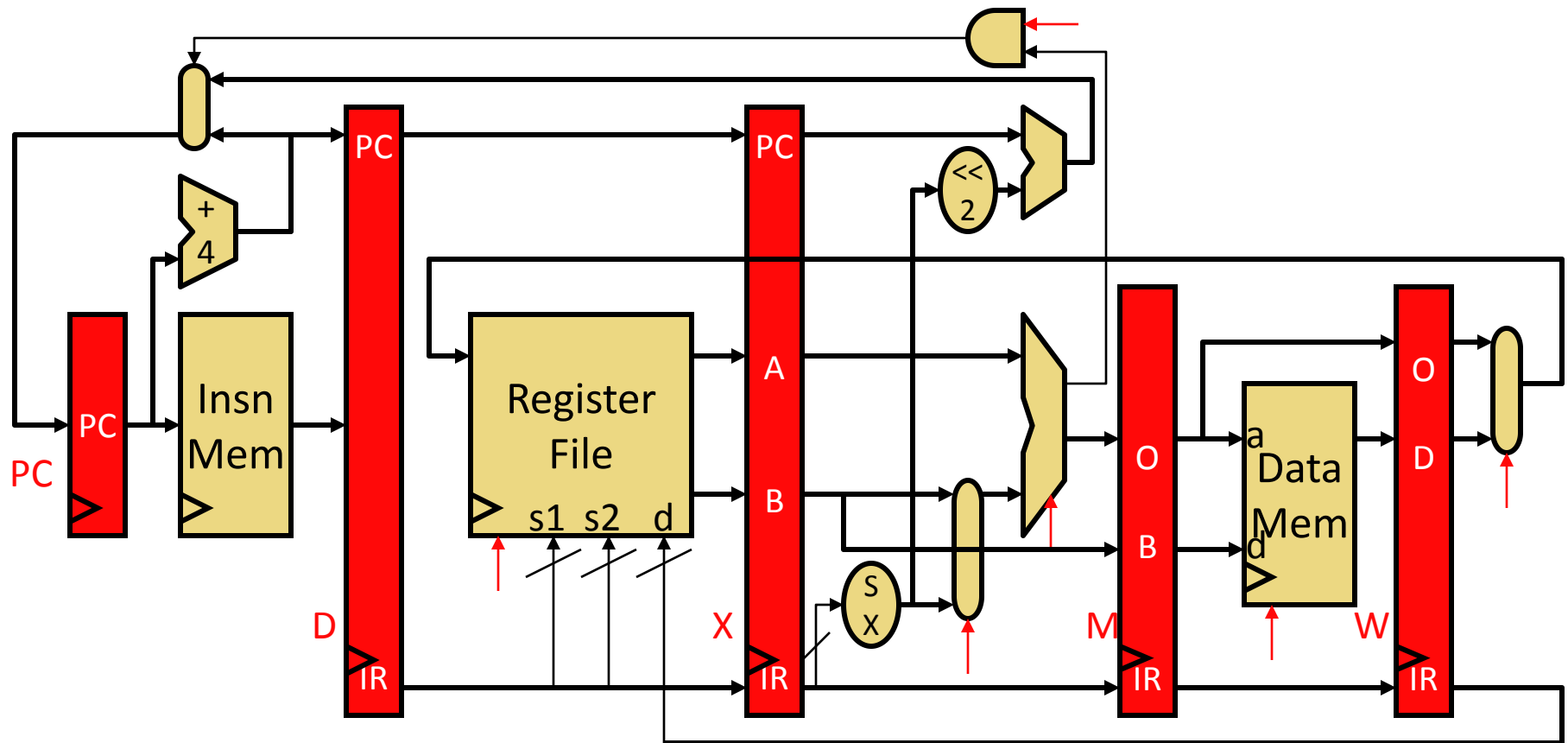
- 每个阶段每个周期中都有一条指令
- + 时钟周期 = $\text{MAX}(T_{\text{insn-mem}}, T_{\text{regfile}}, T_{\text{ALU}}, T_{\text{data-mem}}, T_{\text{writeback}})$
- + 理想 CPI = 1: 每个周期都有1条指令进入和离开 (完成)
- 实际 CPI > 1: 流水线经常会 stall
- 单条指令的执行时间增加了 (pipeline 会引入额外开销)

5级流水线



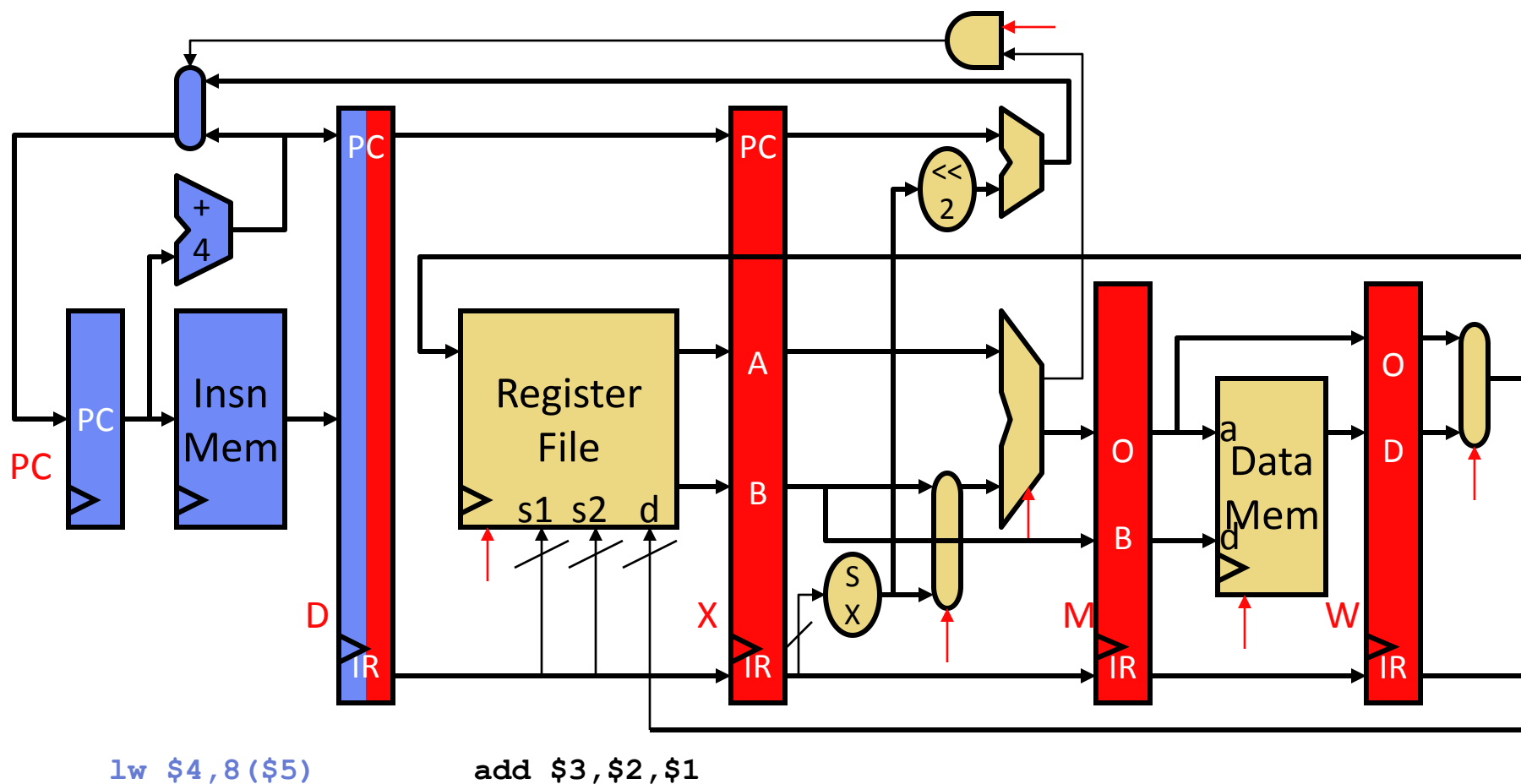
- 5个阶段: **F**etch, **D**ecode, e**X**ecute, **M**emory, **W**riteback
 - 当代CPU流水线级数更多 (Pentium 4 had 22 stages!)
- 流水线寄存器按它们开始的阶段命名
 - **PC, D, X, M, W**

流水线示例: Cycle 1

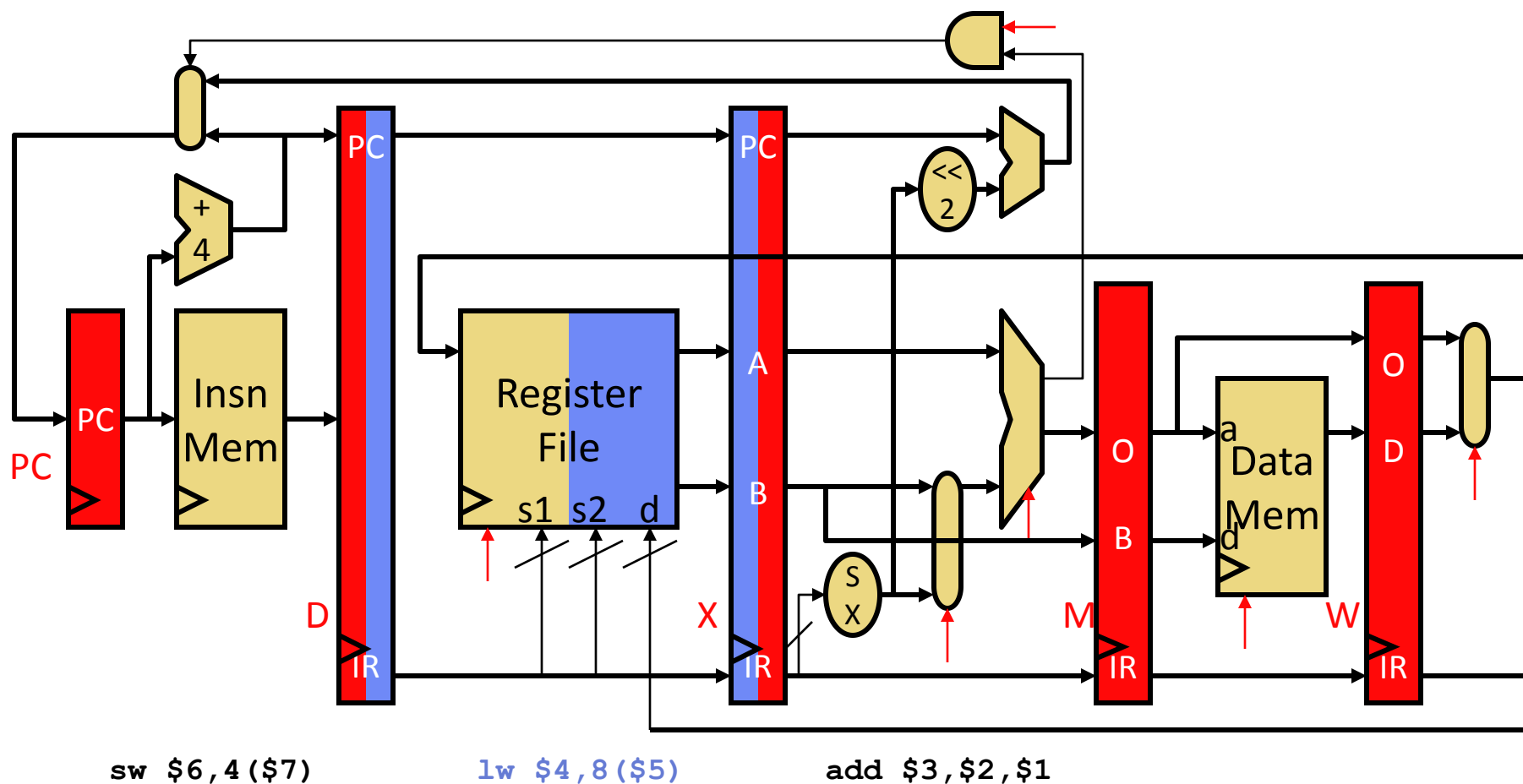


add \$3, \$2, \$1

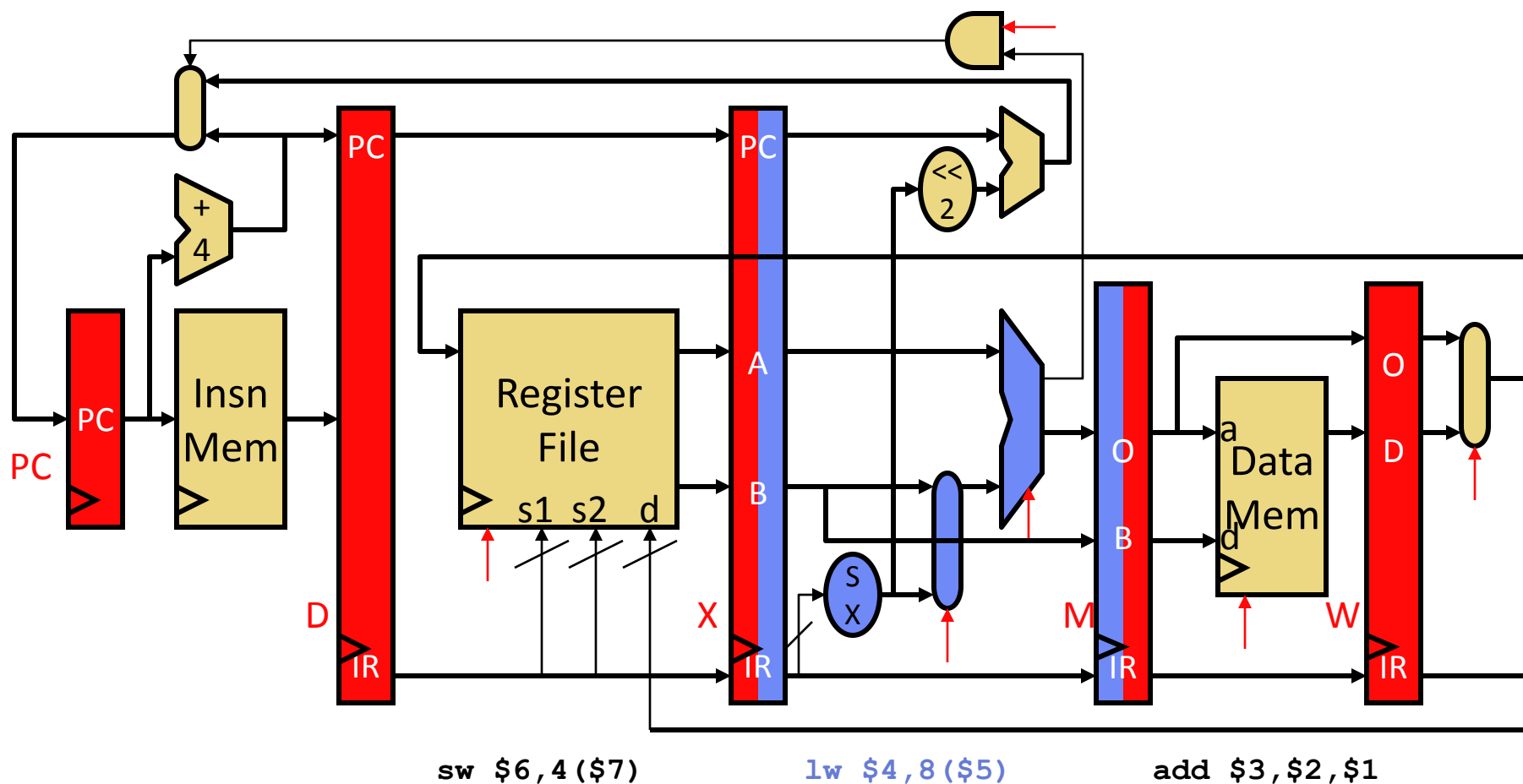
流水线示例: Cycle 2



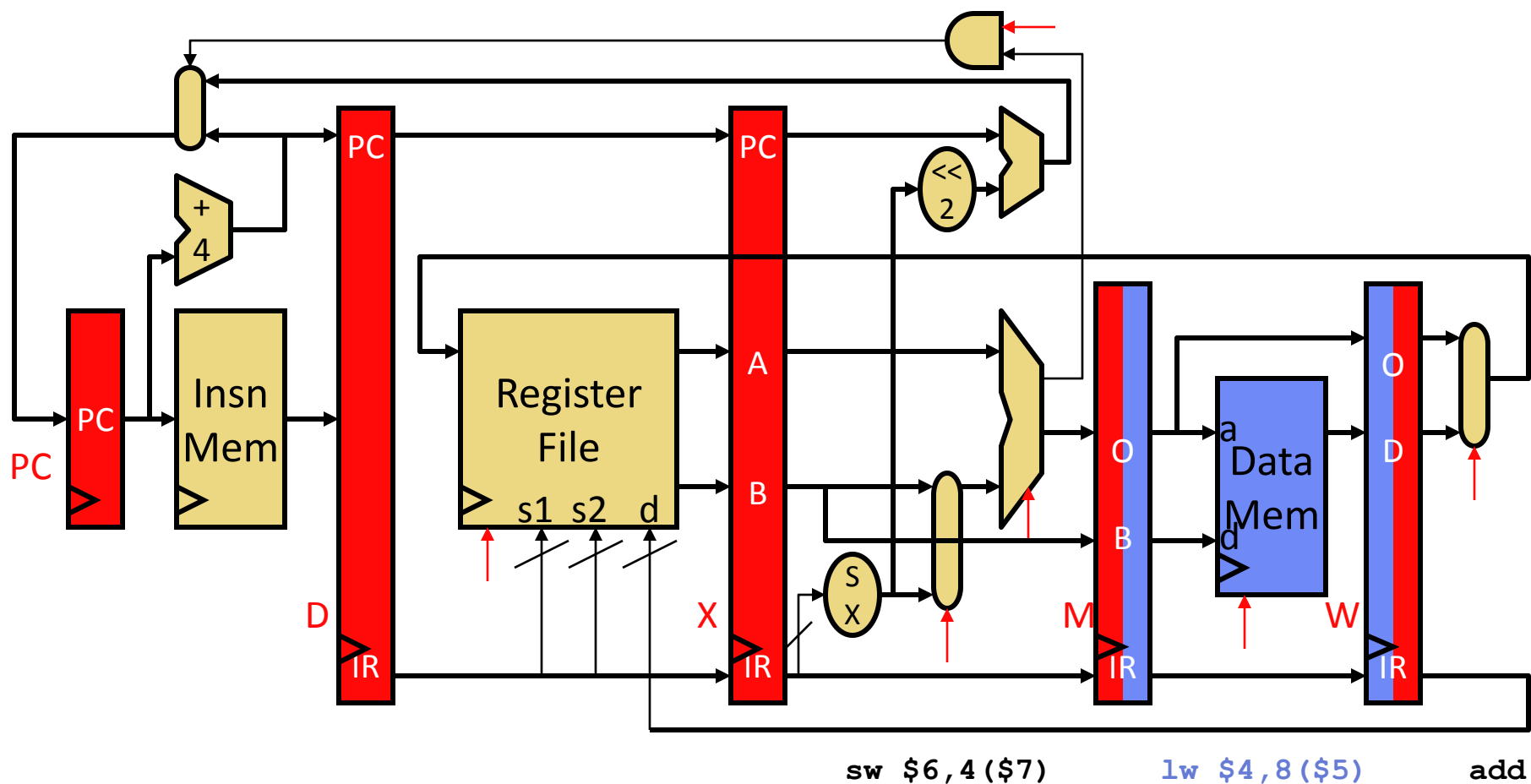
流水线示例: Cycle 3



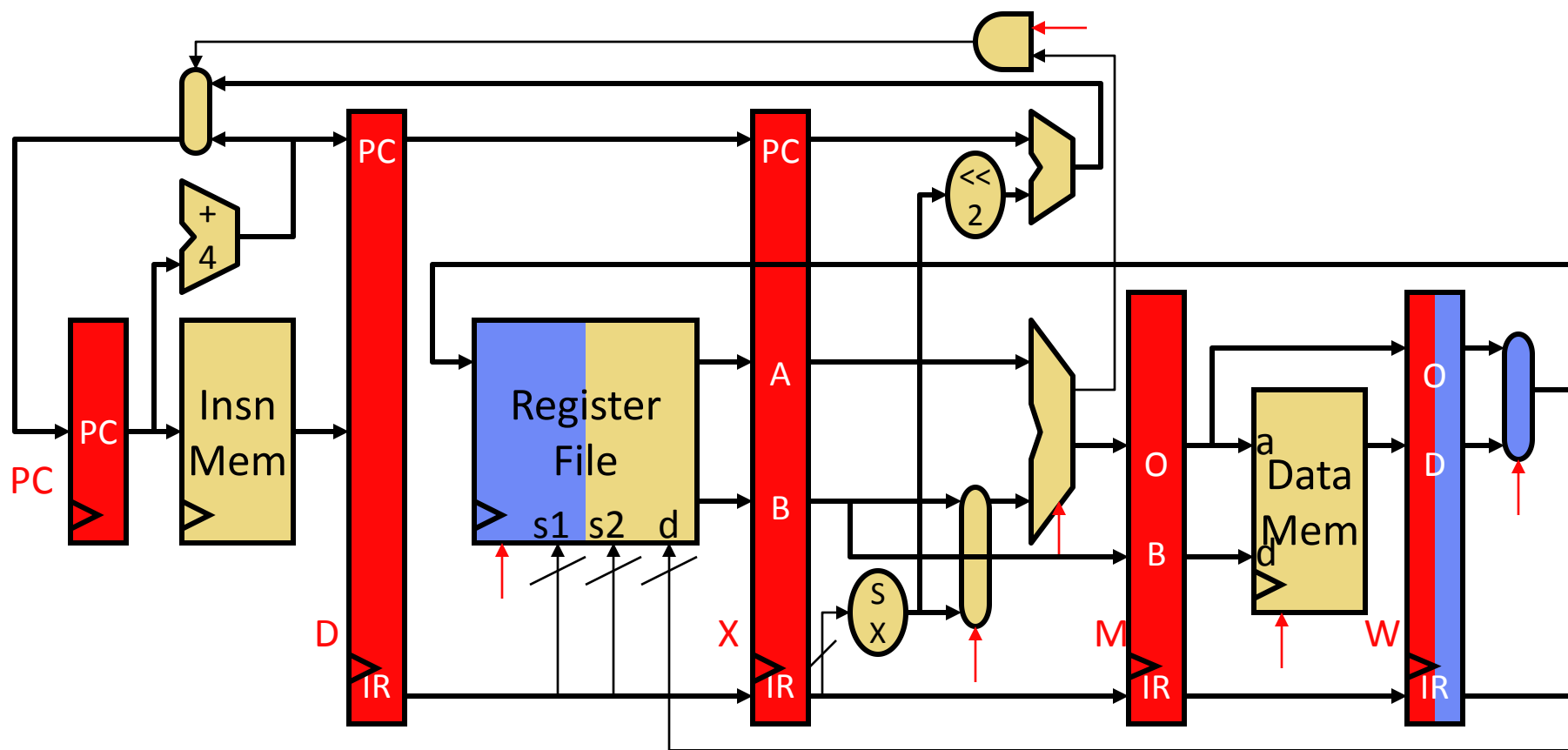
流水线示例: Cycle 4



流水线示例: Cycle 5



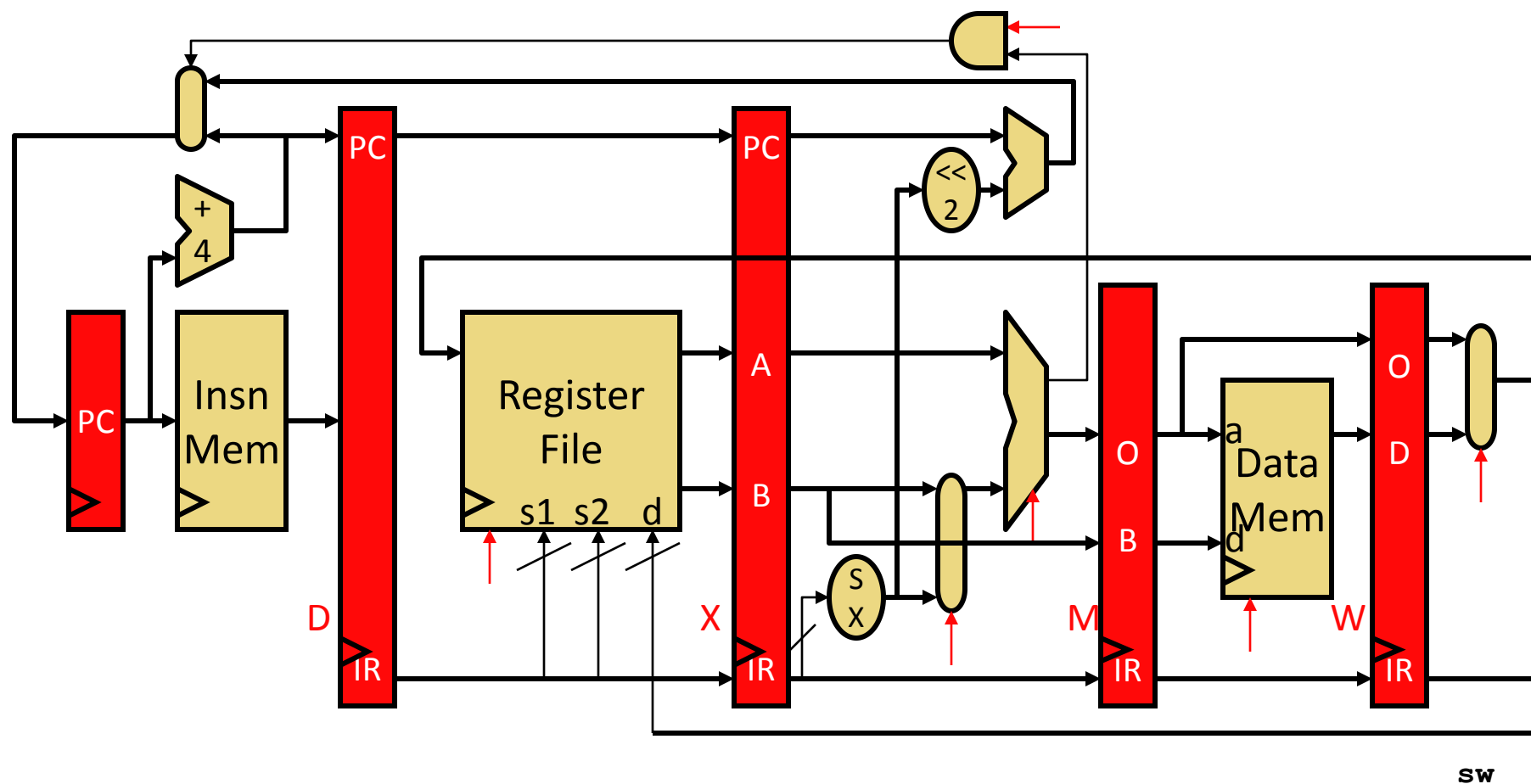
流水线示例: Cycle 6



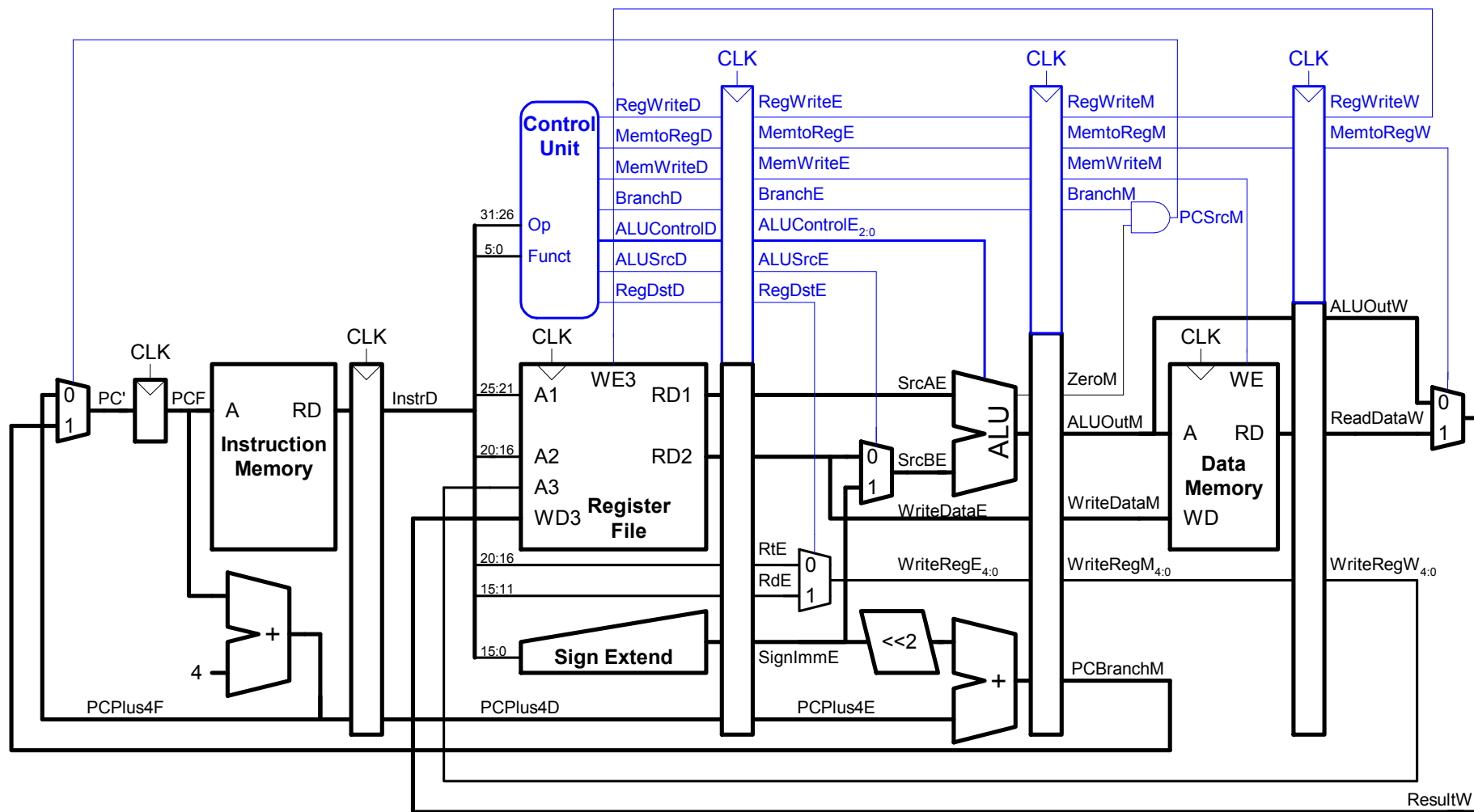
sw \$6, 4 (\$7)

lw

流水线示例: Cycle 7



流水线控制信号



常用方案：在decode阶段产生控制信号，然后向后传递

流水线时序图

□ 时序图规则

- ✓ 横向：周期数
- ✓ 纵向：指令
- ✓ 惯例：**X** 代表指令lw \$4, 8(\$5)在第4周期结束的时候完成了execute阶段
 - 假设没有stall(稍后讨论)

	1	2	3	4	5	6	7	8	9
add \$3,\$2,\$1	F	D	X	M	W				
lw \$4,8(\$5)		F	D	X	M	W			
sw \$6,4(\$7)			F	D	X	M	W		

流水线性能分析-示例

□ 单周期

- ✓ 时钟周期 = 50ns, $CPI = 1$
- ✓ Performance = 50ns/insn

□ 5级流水线

- ✓ 时钟周期 = **12ns** (原因: 50ns/5级 + 额外开销)
- + $CPI = 1$ (每个周期完成1条指令)
- + Performance = **12ns/insn**
- 实际上: CPI通常大于1 (流水线stall)
 - $CPI = 1.5$ (每1.5个周期完成1条指令)
 - Performance = **18ns/insn**
 - 仍然比单周期的性能高很多

关于流水线的时钟周期

为什么流水线的时钟周期 $>$ (单周期的时钟周期) / (流水段个数)?

□ 三个原因

- ✓ 每个流水段都增加了寄存器，增加了延迟
- ✓ 每个流水段的时间长度不同，时钟周期按照最长的计算
- ✓ 流水线还增加了其他线路（前递旁路等）

***这些因素对于理想的流水线阶段数量有重要影响，随着流水段的级数增加，时钟频率的增益会减少（overhead更高）**

关于流水线的CPI

为什么流水线的CPI > 1?

- 对于简单流水线 (scalar in-order pipeline) , **CPI = 1**
+ stall惩罚
- Stalls是为了解决流水线中的危害 (hazards)
 - **Hazard:** 影响流水线正确执行的一些危害 (数据相关或者控制相关)
 - **Stall:** 为了让流水线正确执行引发的停顿

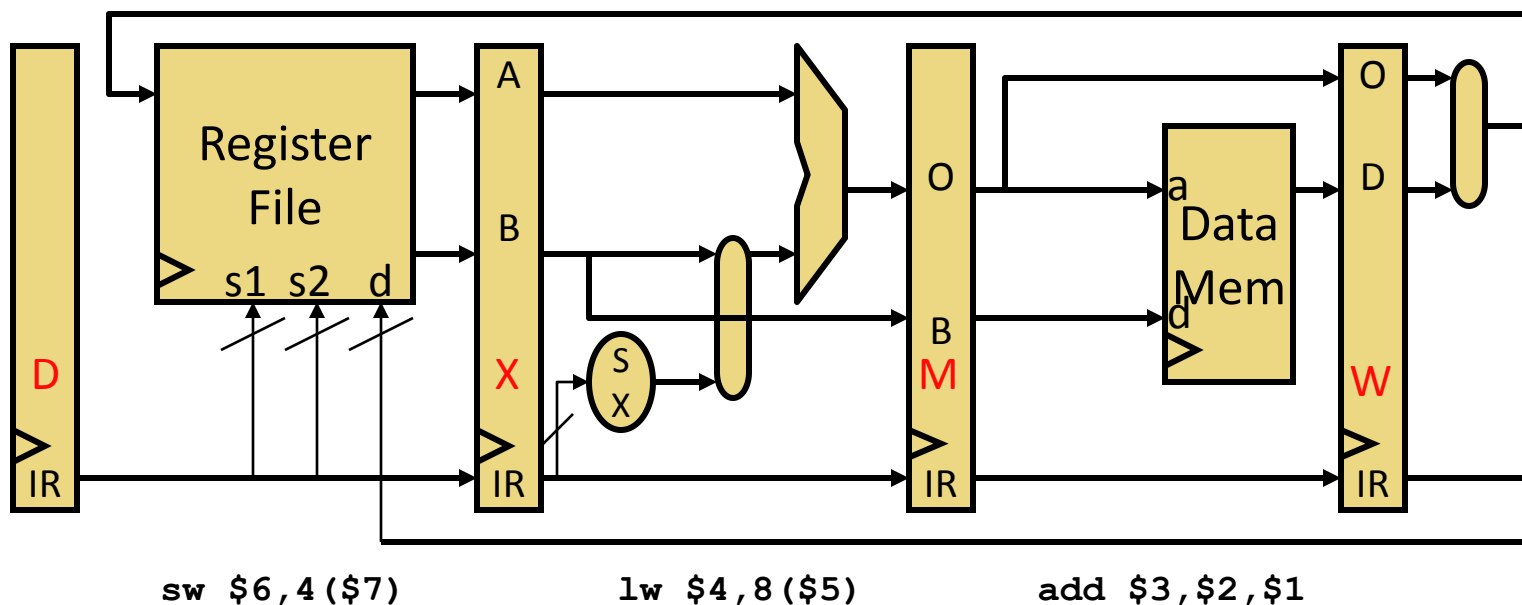
截至目前，流水线：

- **Scalar pipeline**: 每个周期每个阶段1条指令
- **In-order pipeline**: 指令按顺序经过每个阶段
- **指令无数据相关性**
- **无分支指令**
- **所有指令单个周期完成**
 - 浮点，乘法/除法，访存操作

指令相关性及危害

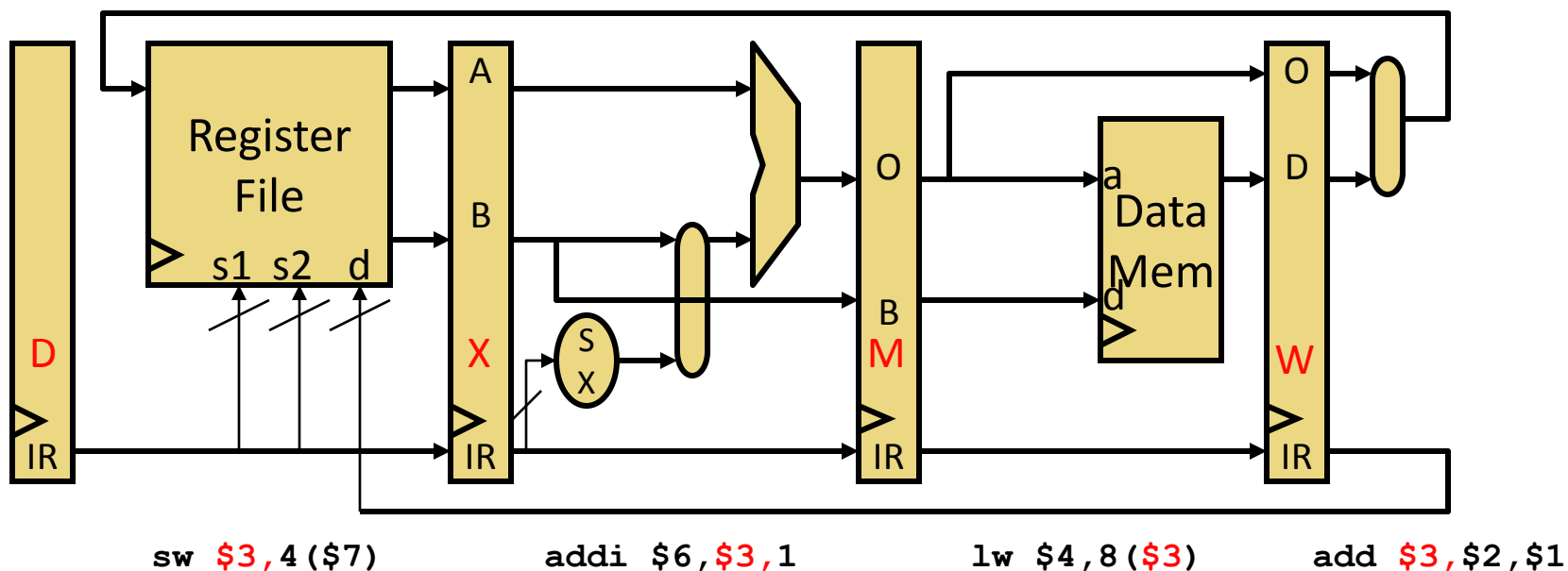
(Dependence and Hazards)

指令相关性



- 假设不考虑条件分支语句
- 上面3条指令能够正确执行（不存在任何相关性）
 - ✓ 但这不是真实程序，真实程序存在相关性

指令相关性



• 上述指令能正确执行么？

- ✓ **add** 在本周期正在将结果写回 **\$3**
- **lw** 在2个周期之前读取 **\$3** → 读取的内容不正确
- **addi** 在1个周期之前读取 **\$3** → 读取的内容不正确
- ✓ **sw** 在本周期正在读取 **\$3** → 读取的内容可能不正确（取决于寄存器的具体实现技术）

指令相关性及危害

□ **指令相关性**: 两条指令之间存在关联关系

- ✓ **数据相关性**: 两条指令使用相同的存储位置
 - 读后写(WAR), 写后读(RAW), 写后写(WRW)
- ✓ **控制相关性**: 一条指令影响另一条指令是否能够执行
 - 分支语句, 中断
- ✓ **结构相关**: 两条指令使用同一种流水线资源

□ **危害 (Hazard)**: 有些相关性会导致指令执行错误

- ✓ 例: 一条指令以前一条指令的结果作为操作数, 但前一条指令的结果尚未写入寄存器或内存, 导致结果错误

数据相关性

数据相关类型

- 写后读 (RAW, 真相关)
- 写后写 (WAW)
- 读后写 (WAR)

RAW (写后读)

mul r0 * r1 → **r2**

...

add **r2** + r3 → r4

WAW (写后写)

mul r0 * r1 → **r2**

...

add r1 + r3 → **r2**

WAR (读后写)

mul r0 * **r1** → r2

...

add r3 + r4 → **r1**

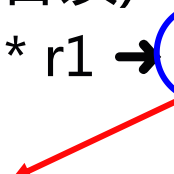
数据相关性

哪些数据相关性会导致流水线停顿？

- ✓ “写后读” 是对某个值的真正依赖，必须解决
- ✓ “写后写” 和 “读后写” 是因为寄存器数量不够引起的
 - 它们依赖某个寄存器 “名字”，而不是一个值
 - 后面会讨论如何解决

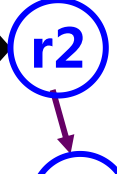
RAW (写后读)

mul r0 * r1 → **r2**
...
add **r2** + r3 → r4



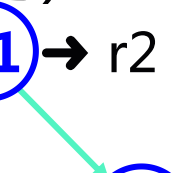
WAW (写后写)

mul r0 * r1 → **r2**
...
add r1 + r3 → **r2**



WAR (读后写)

mul r0 * **r1** → r2
...
add r3 + r4 → **r1**



解决数据相关性 (RAW)

Strategy 1: 暂停流水线，直到计算出结果(stall)

Strategy 2: 一旦数据计算完成，尽快将数据前递到先前的流水线阶段 (bypass)

解决数据相关性 (RAW)

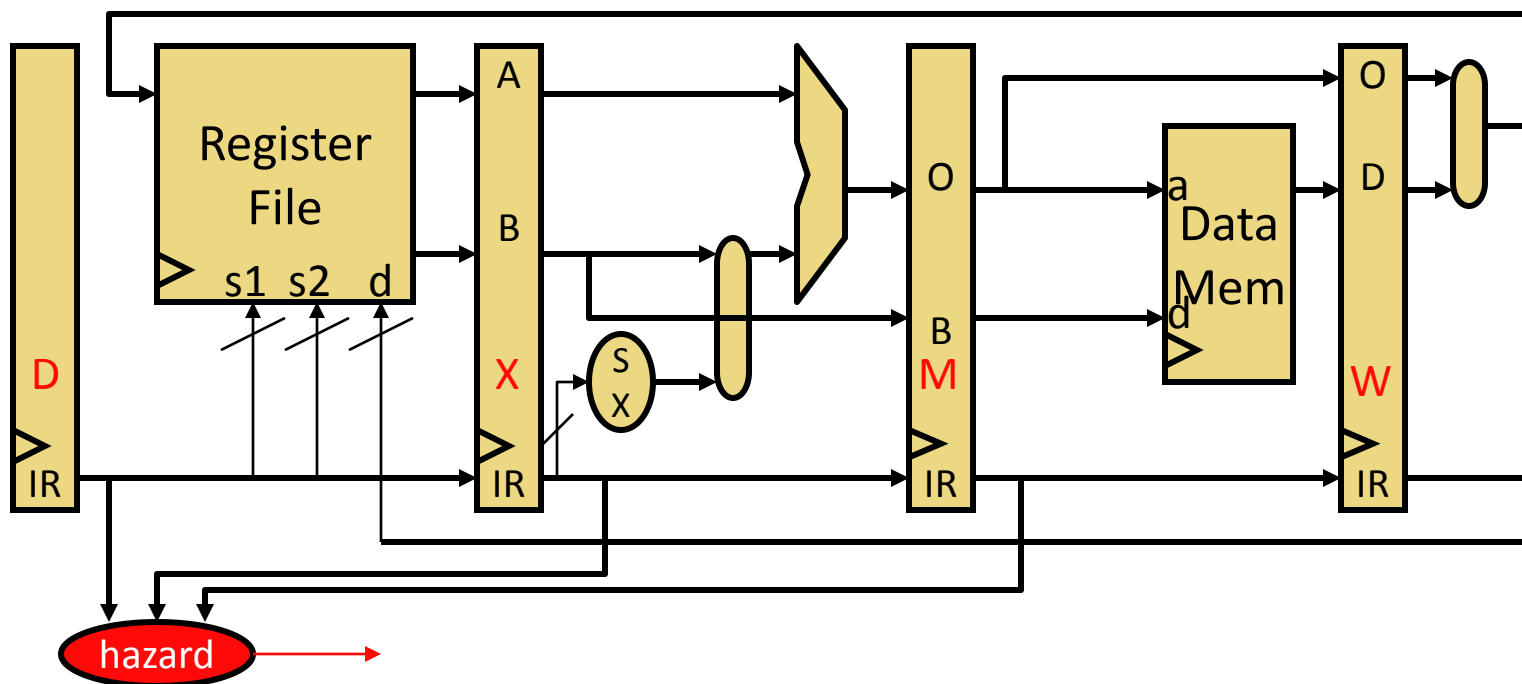
Strategy 1: 暂停流水线，直到计算出结果(stall)

- 处理器检测数据危害并将其消除
- 不存在兼容性的问题
- 两个方面：
 - 检测数据危害
 - 消除数据危害

□ 采用组合电路

- ✓ 检测流水线后面阶段的指令是否要写当前正在译码阶段指令的源寄存器
 - Yes: stall
 - No: 无需stall... 没有相关性

检测数据危害

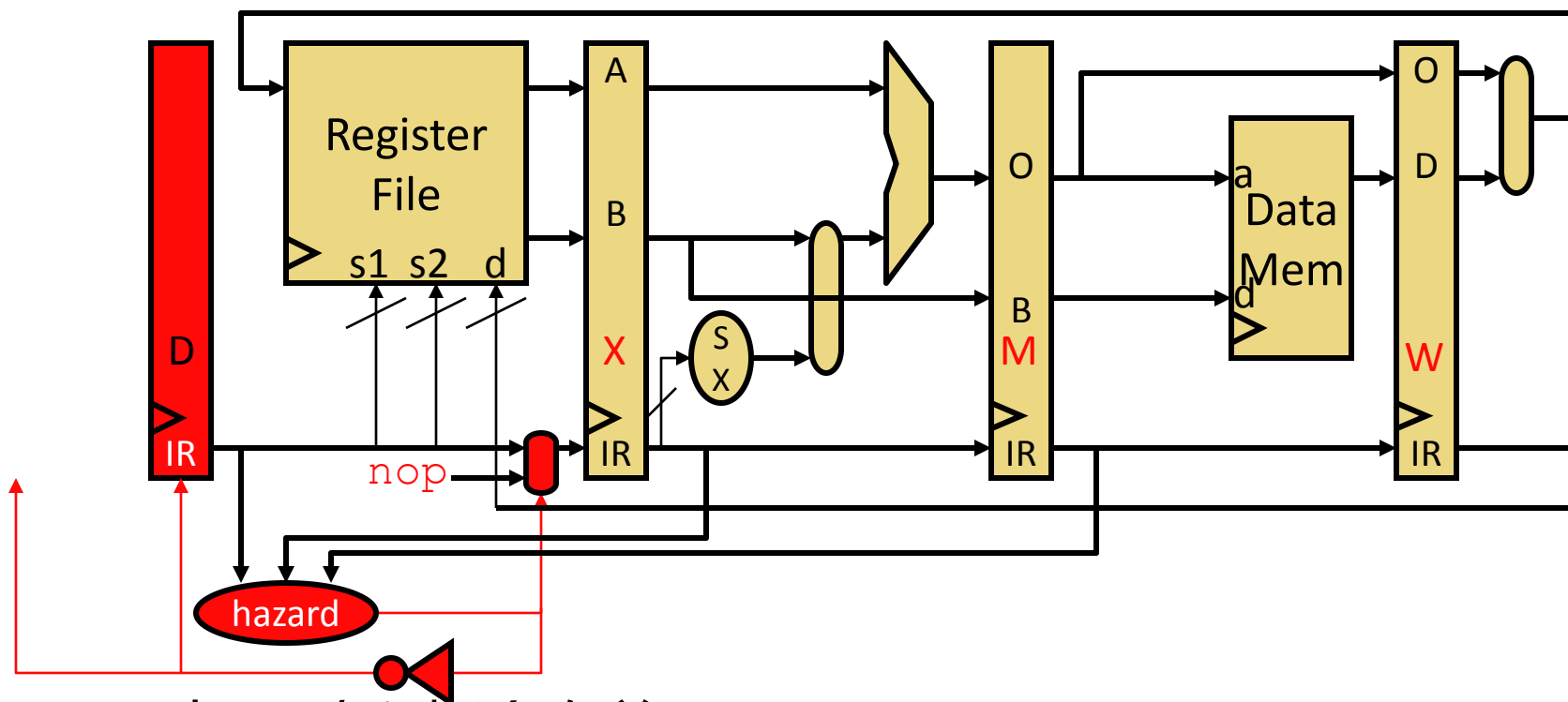


- 比较D阶段指令的源寄存器和流水线前面指令的目标寄存器

Stall =

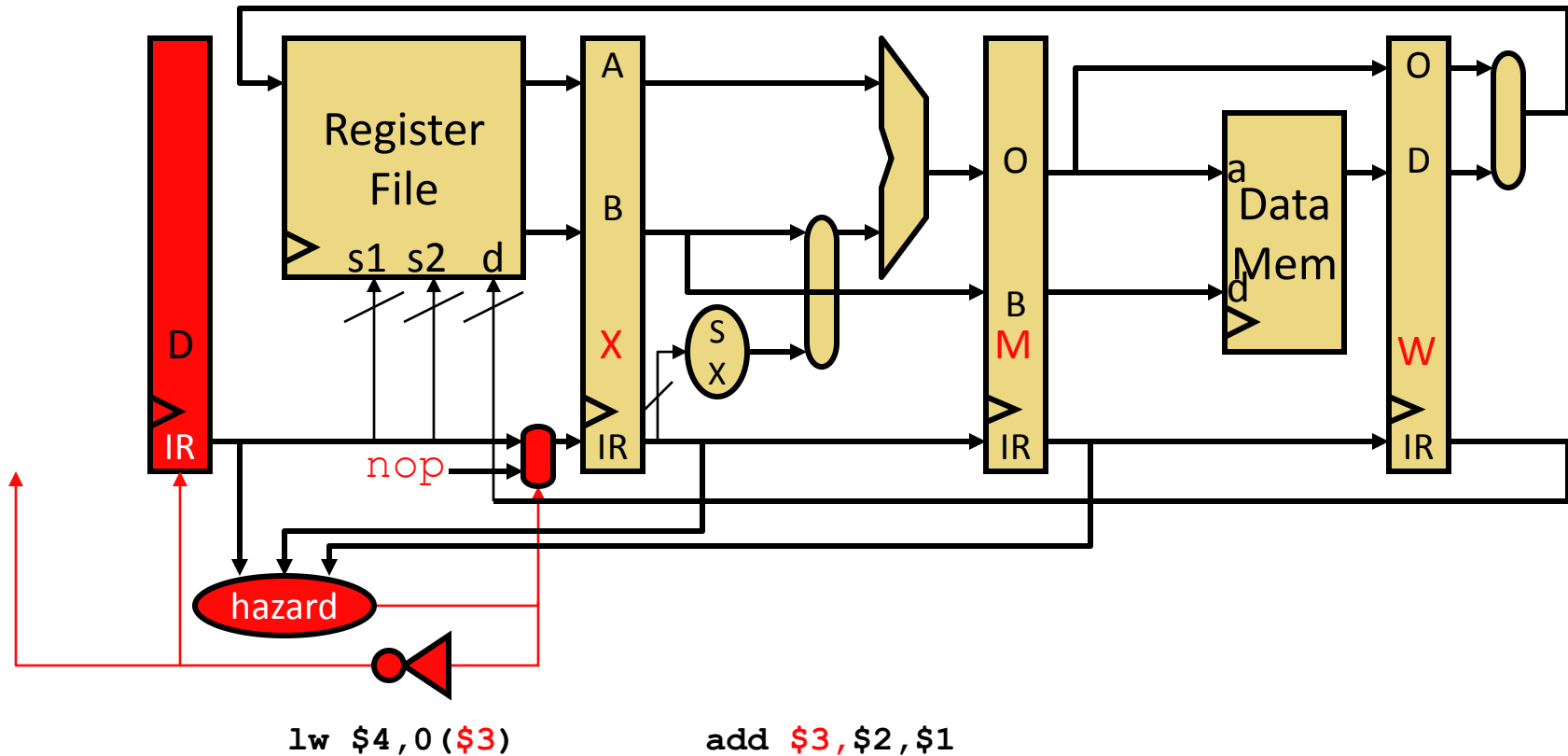
```
(D.IR.RegSrc1 == X.IR.RegDest) ||  
(D.IR.RegSrc2 == X.IR.RegDest) ||  
(D.IR.RegSrc1 == M.IR.RegDest) ||  
(D.IR.RegSrc2 == M.IR.RegDest)
```

检测数据危害



- 阻止 D 阶段指令向前
 - ✓ 向X.IR写入 **nop** (插入空指令，将无效控制信号送入X.IR)
 - ✓ 将D阶段寄存器的write enable置为无效（保证本条指令状态不被下一条指令状态覆盖）
 - ✓ 将PC的write enable置为无效（保证新的指令不会进入流水线）
- 下个周期重新检测

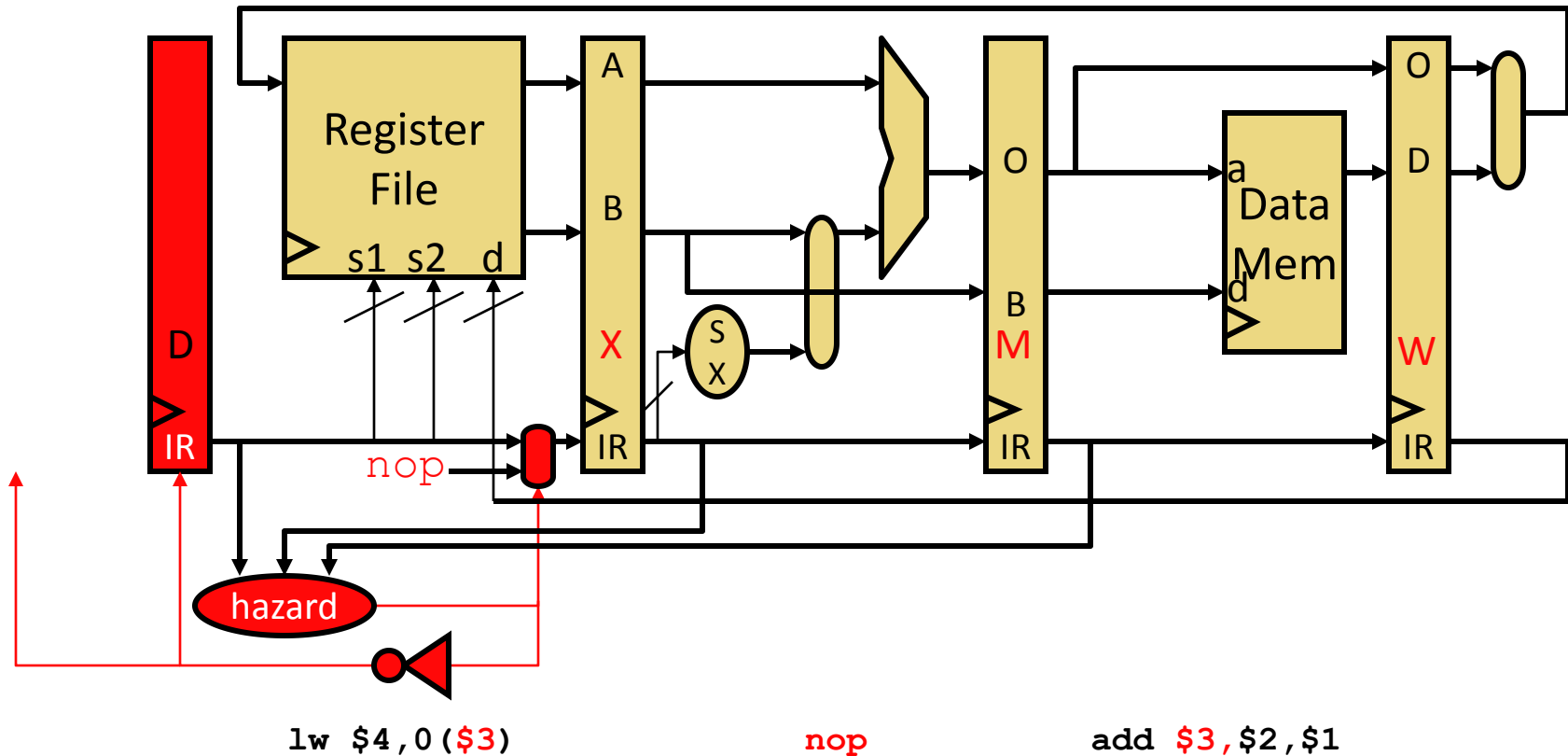
示例 : Cycle 1



Stall =

```
(D.IR.RegSrc1 == X.IR.RegDest) ||
(D.IR.RegSrc2 == X.IR.RegDest) ||
(D.IR.RegSrc1 == M.IR.RegDest) ||
(D.IR.RegSrc2 == M.IR.RegDest) = 1
```

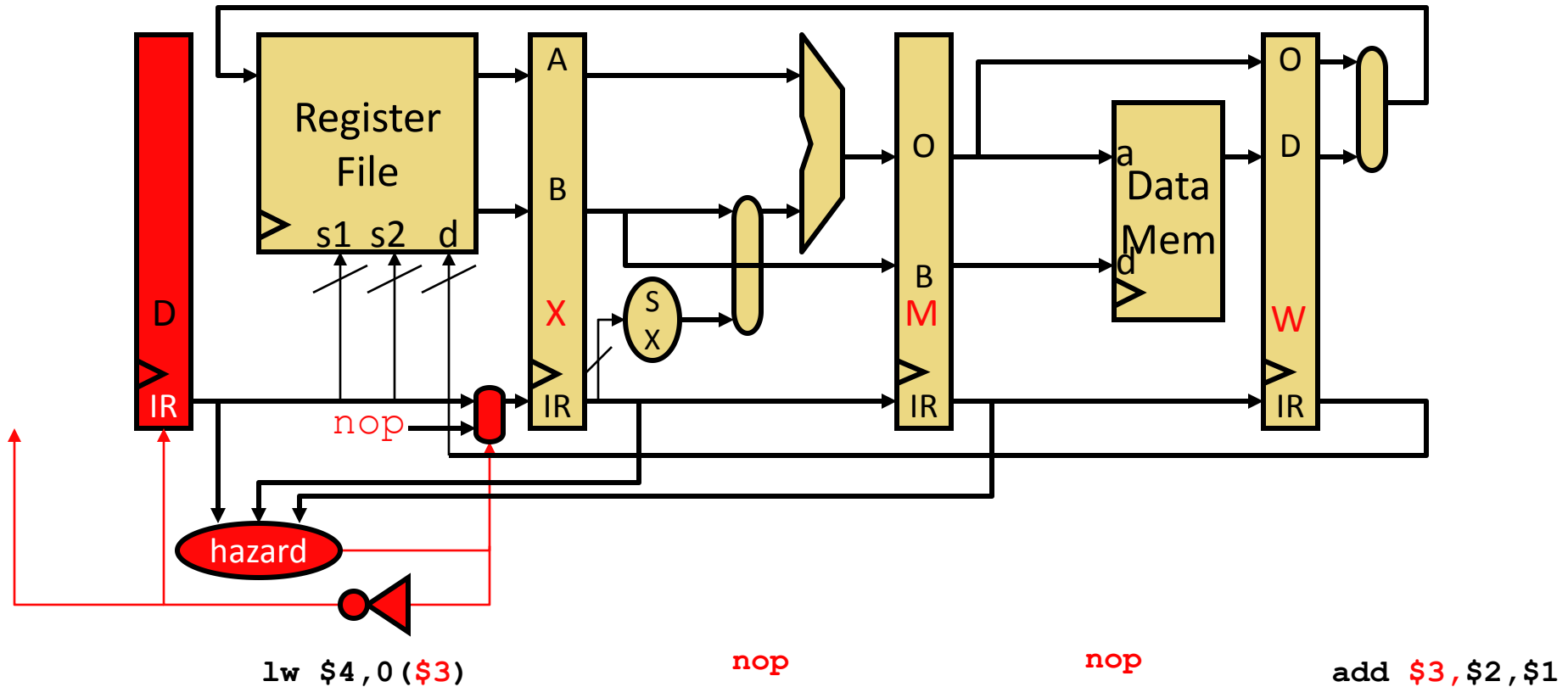
示例 : Cycle 2



Stall =

```
(D.IR.RegSrc1 == X.IR.RegDest) ||
(D.IR.RegSrc2 == X.IR.RegDest) ||
(D.IR.RegSrc1 == M.IR.RegDest) ||
(D.IR.RegSrc2 == M.IR.RegDest) = 1
```

示例 : Cycle 3



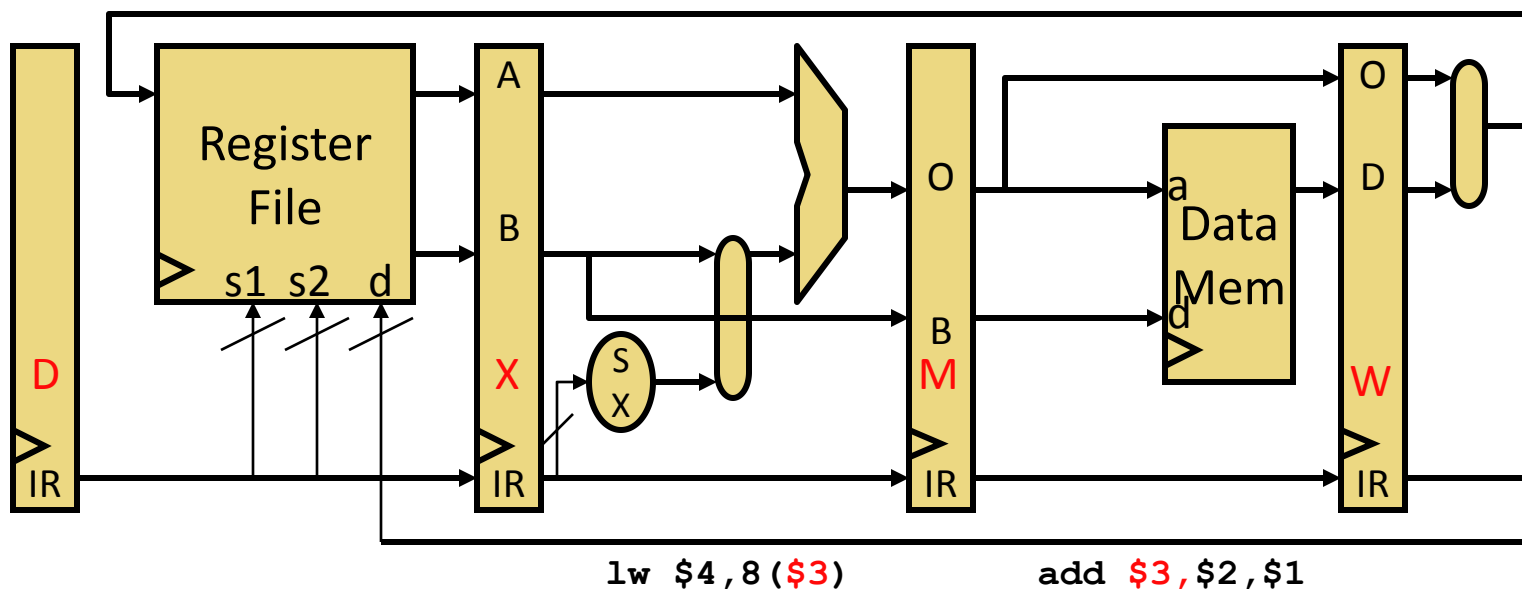
Stall =

```
(D.IR.RegSrc1 == X.IR.RegDest) ||
(D.IR.RegSrc2 == X.IR.RegDest) ||
(D.IR.RegSrc1 == M.IR.RegDest) ||
(D.IR.RegSrc2 == M.IR.RegDest) = 0
```

解决数据相关性 (RAW)

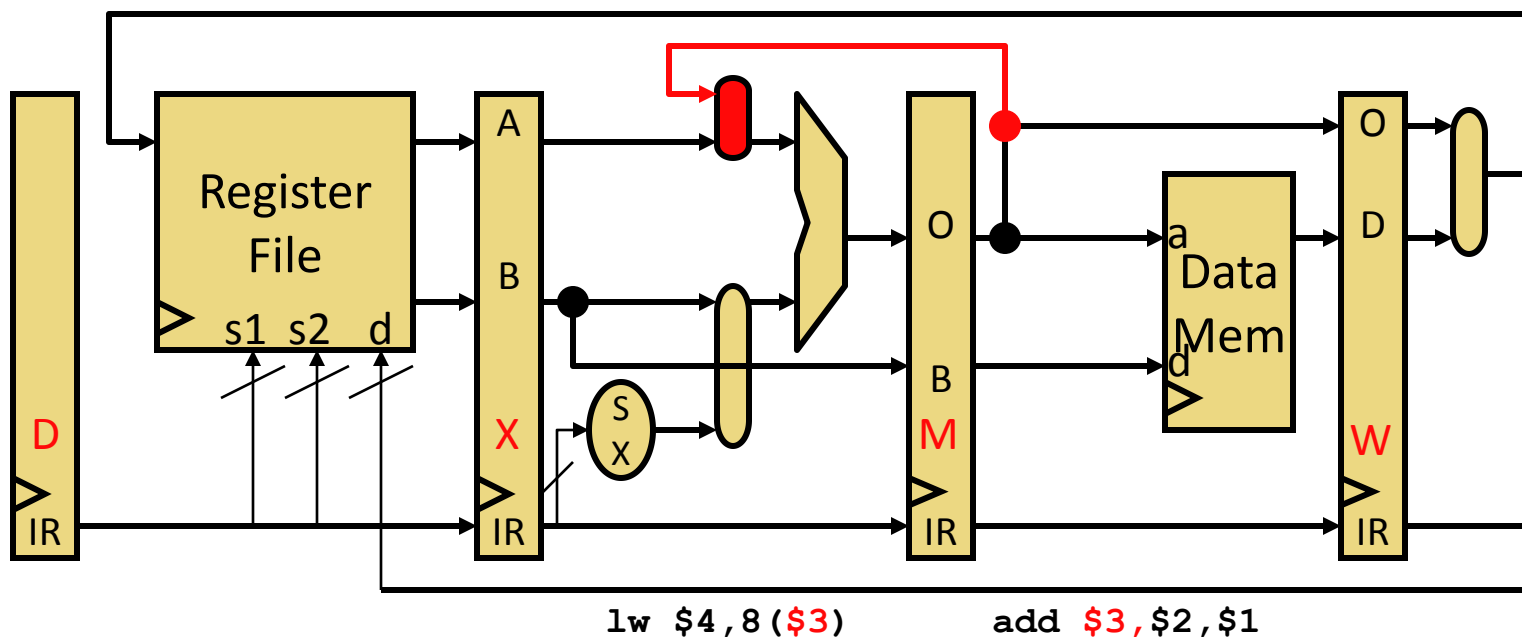
Strategy 2: 一旦数据计算完成，尽快将数据前递到先前的流水线阶段 (bypass)

观察



- 看上去，上述指令会产生数据危害
 - ✓ `lw $4, 8($3)` 已经读取了 `$3`
 - ✓ `add $3, $2, $1` 尚未将结果写回 `$3`
- 实际上，一切都还来得及！
 - ✓ `lw $4, 8($3)` 还未使用 `$3`
 - ✓ `add $3, $2, $1` 已经计算完了 `$3`

旁路 (Bypassing)

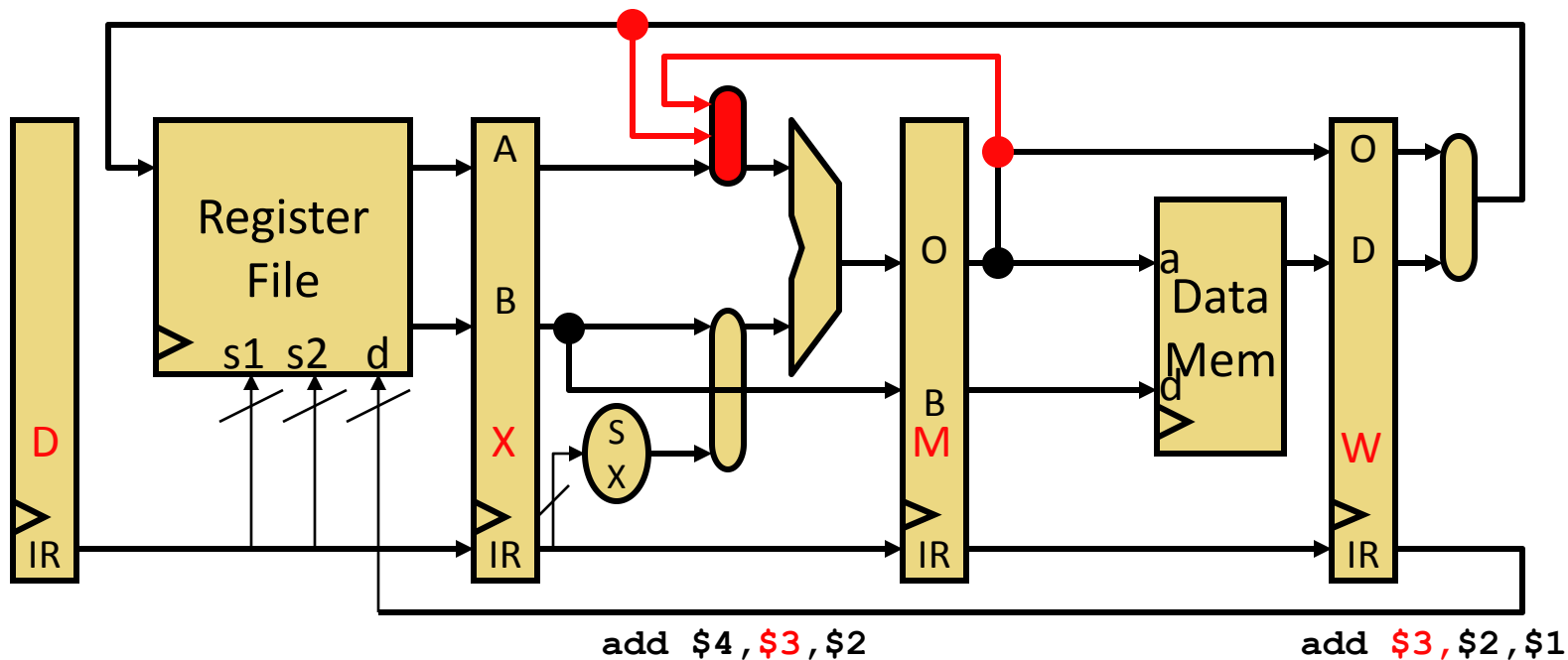


- Bypassing**

- ✓ 从指令执行的中间结果读取数据（增加从M阶段到X阶段的旁路）
- ✓ 不用等到最终计算结果写回（直接将add的计算结果送到X阶段）
- ✓ 也称为“前递” (forwarding)

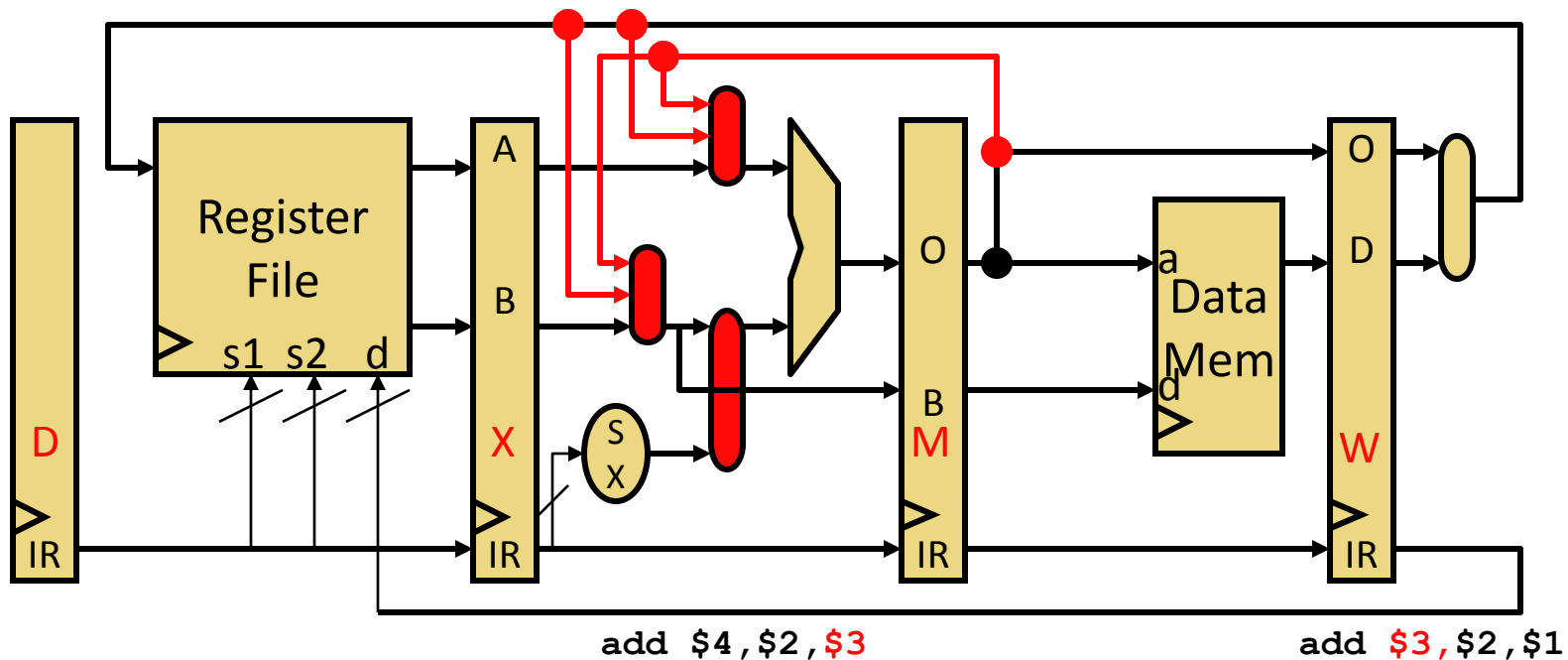
注意：这里只对寄存器的值进行旁路

WX Bypassing



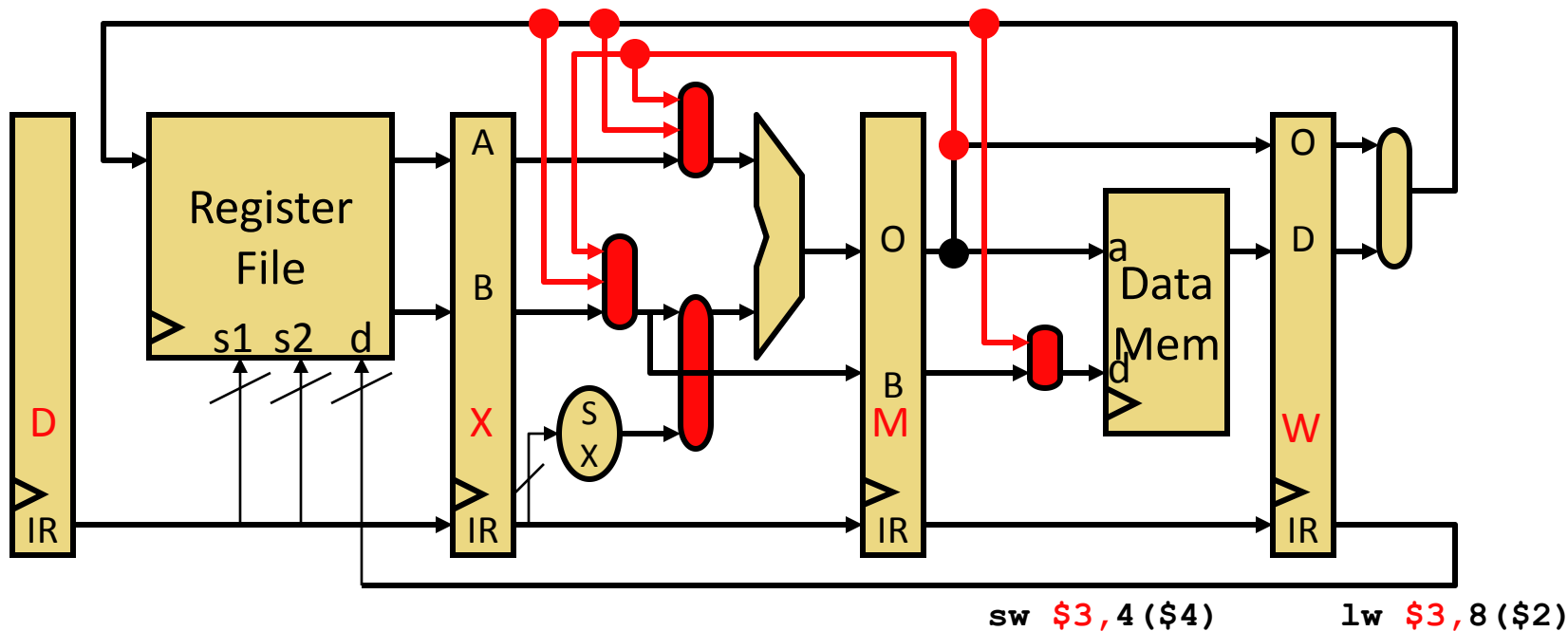
- 增加W->X的旁路
 - ✓ X阶段需要增加一个选择器，选择使用从M或W阶段传递过来的值

Bypass到ALU的另一个输入



- 也可以同时bypass到ALU的另一个输入

WM Bypassing?



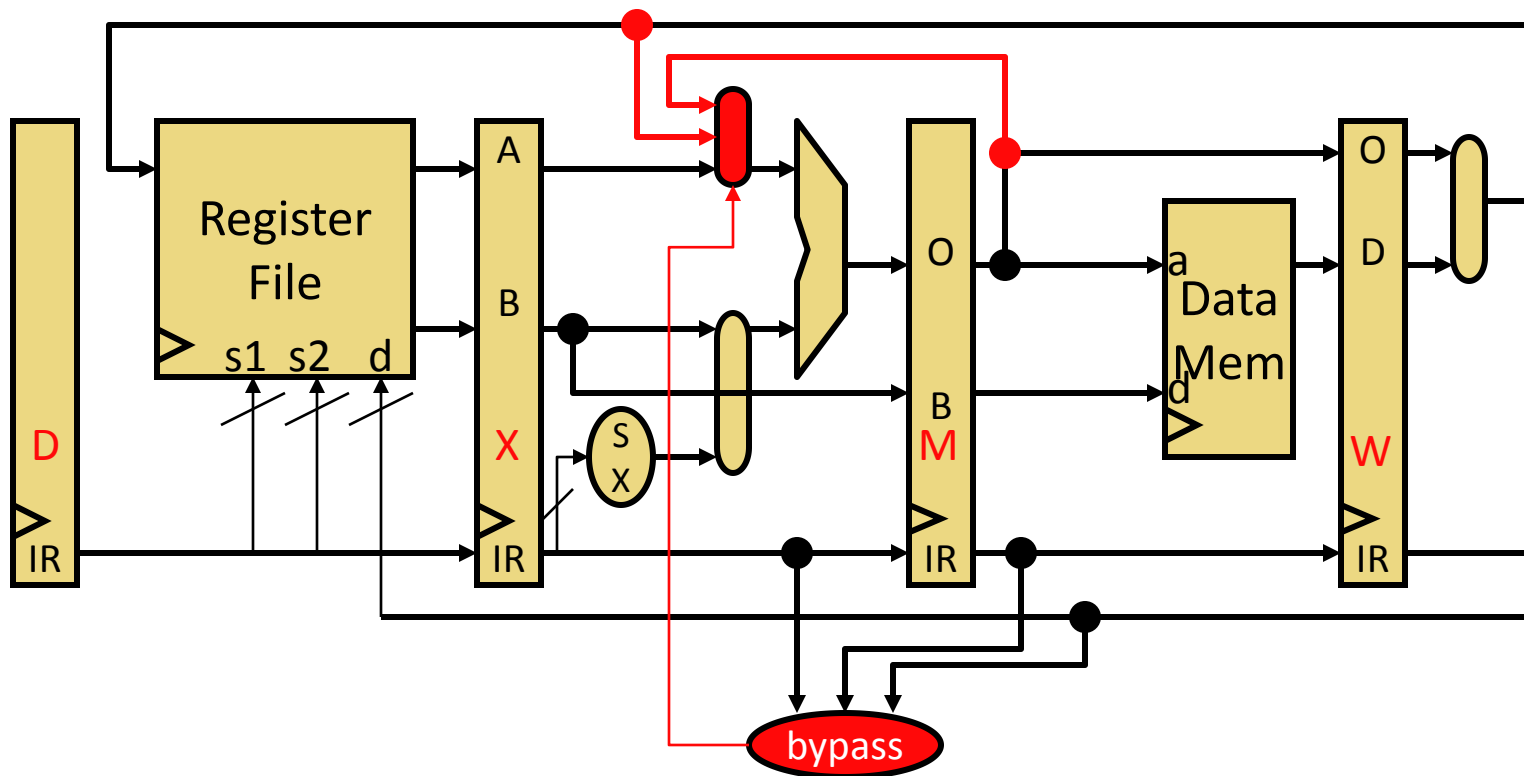
- W → M 的bypassing能解决下面的相关性么？
 - ✓ 不能解决 (sw在X阶段就需要\$3，必须stall一个周期)

sw \$4, 4 (\$3) lw \$3, 8 (\$2)

- ✓ 能解决 (sw在M阶段才需要\$3)

sw \$3, 4 (\$4) lw \$3, 8 (\$2)

Bypass 的控制逻辑



- 每个多路选择器有自己的控制逻辑
 - ✓ 例：控制ALU的A输入的控制逻辑
$$(X.IR.RegSrc1 == M.IR.RegDest) \Rightarrow 0$$
$$(X.IR.RegSrc1 == W.IR.RegDest) \Rightarrow 1$$
$$\text{Else} \Rightarrow 2$$

带有bypass的流水线时序图

- Example: MX bypass

	1	2	3	4	5	6	7	8	9	10
add r1, r2, r3	F	D	X	M	W					
sub r2, r1, r4		F	D	X	M	W				

- Example: WX bypass

	1	2	3	4	5	6	7	8	9	10
add r1, r2, r3	F	D	X	M	W					
lw r5, 4(r7)		F	D	X	M	W				
sub r2, r1, r4			F	D	X	M	W			

- Example: WM bypass

	1	2	3	4	5	6	7	8	9	10
add r1, r2, r3	F	D	X	M	W					
?		F	D	X	M	W				

你能想一个例子（添加一条指令），使用WM bypass么？

Bypass 和 Stall 对比

□ 两种不同思路

- Stall逻辑控制流水线寄存器（该不该插入空指令）
- Bypass逻辑控制多路选择器（该不该用bypass过来的值）

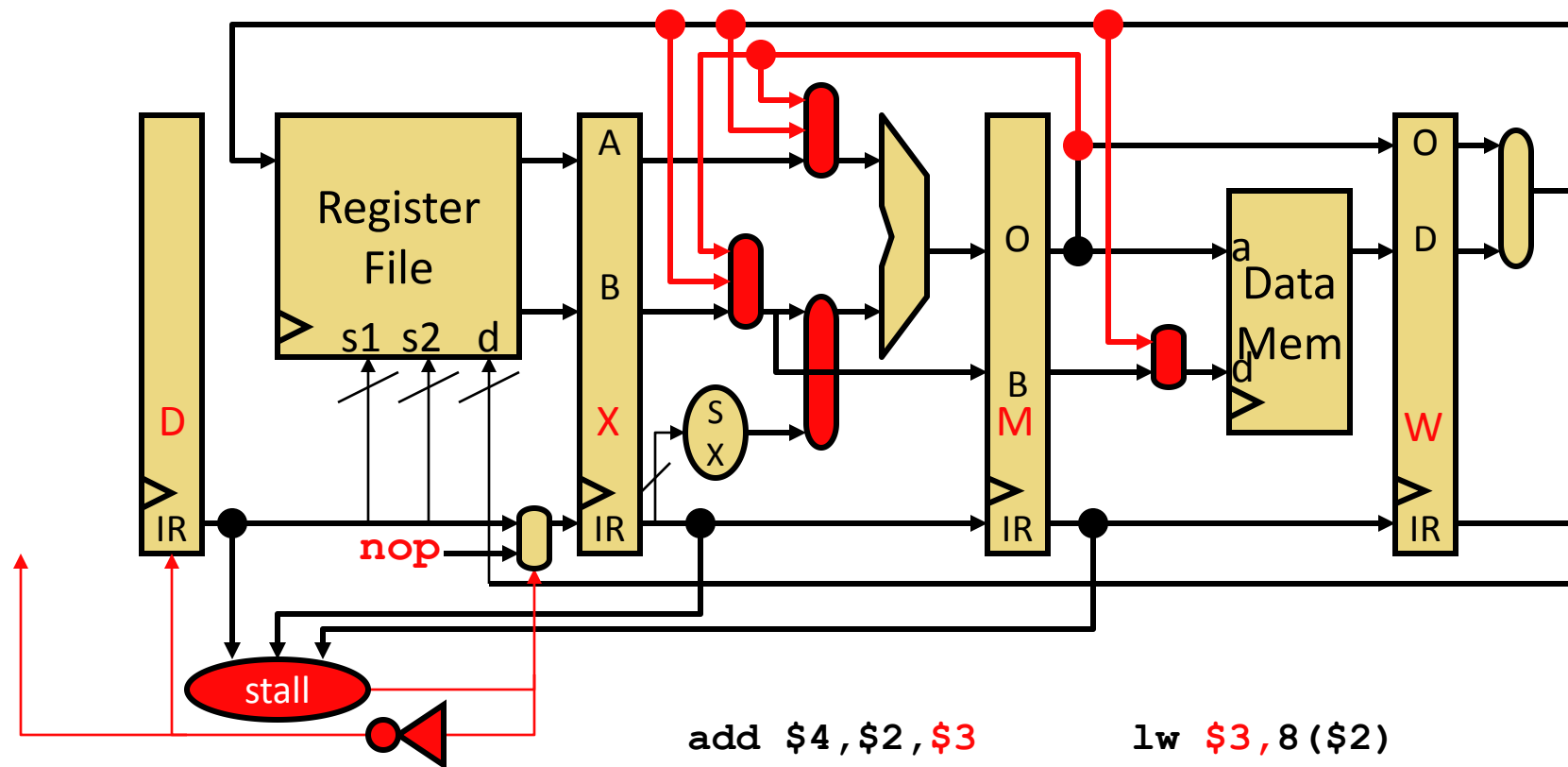
□ 两者互补

- 遇到数据危害: 如果不能bypass, 就必须stall

□ 前面的slides展示的是**full bypassing**: 所有阶段都可能bypasses到X阶段

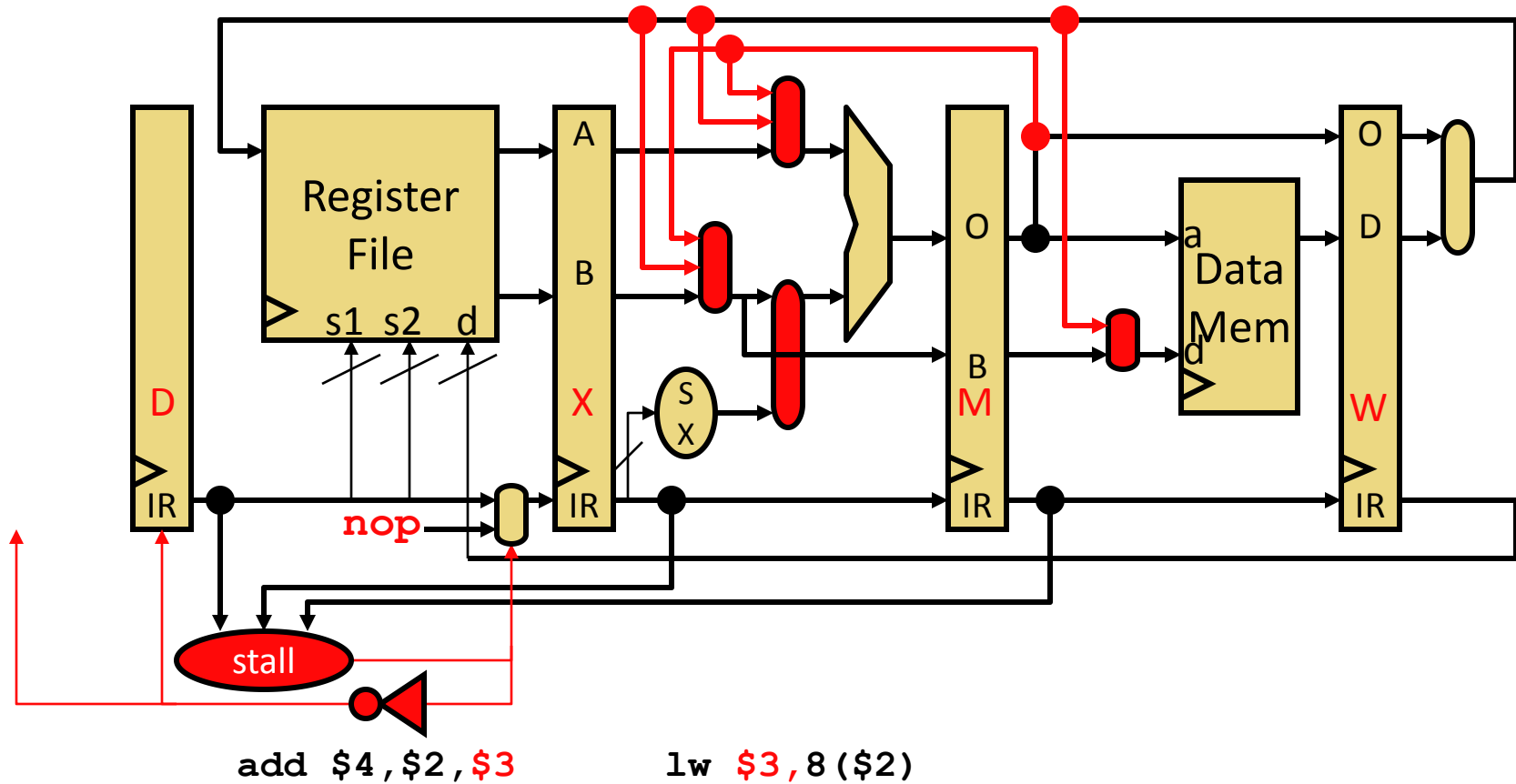
- 能解决所有的data hazards么？

Bypass能解决所有的数据危害么?



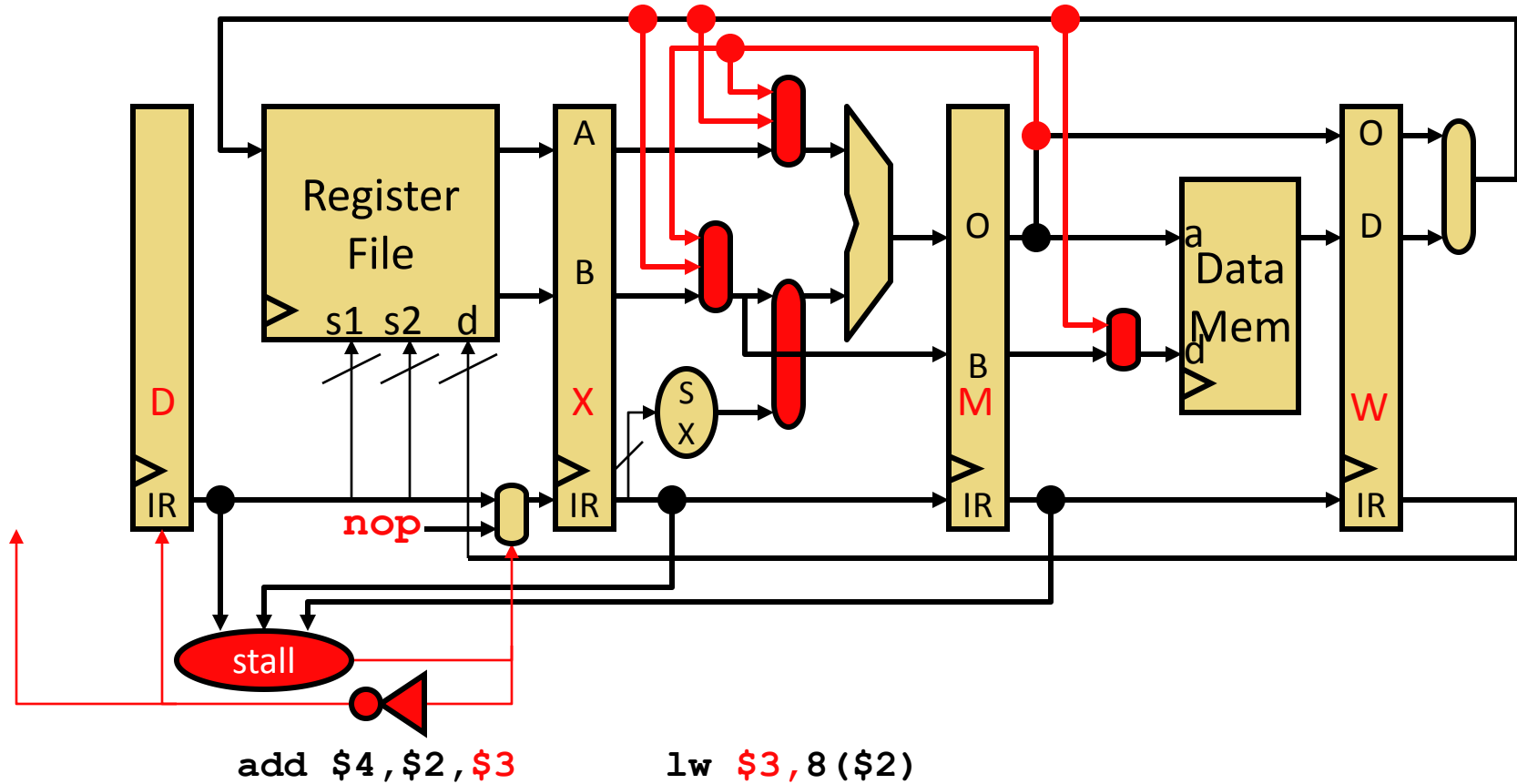
- 不能：考虑 “load” 后面紧跟着 “add”
- load在M阶段结束后才能获得\$3, add在X阶段就需要\$3!
- 硬件方法: 检测这种情况并插入stall
- 软件方法: 确保编译器不生成这样的代码

硬件方法（解决Load-To-Use相关性）



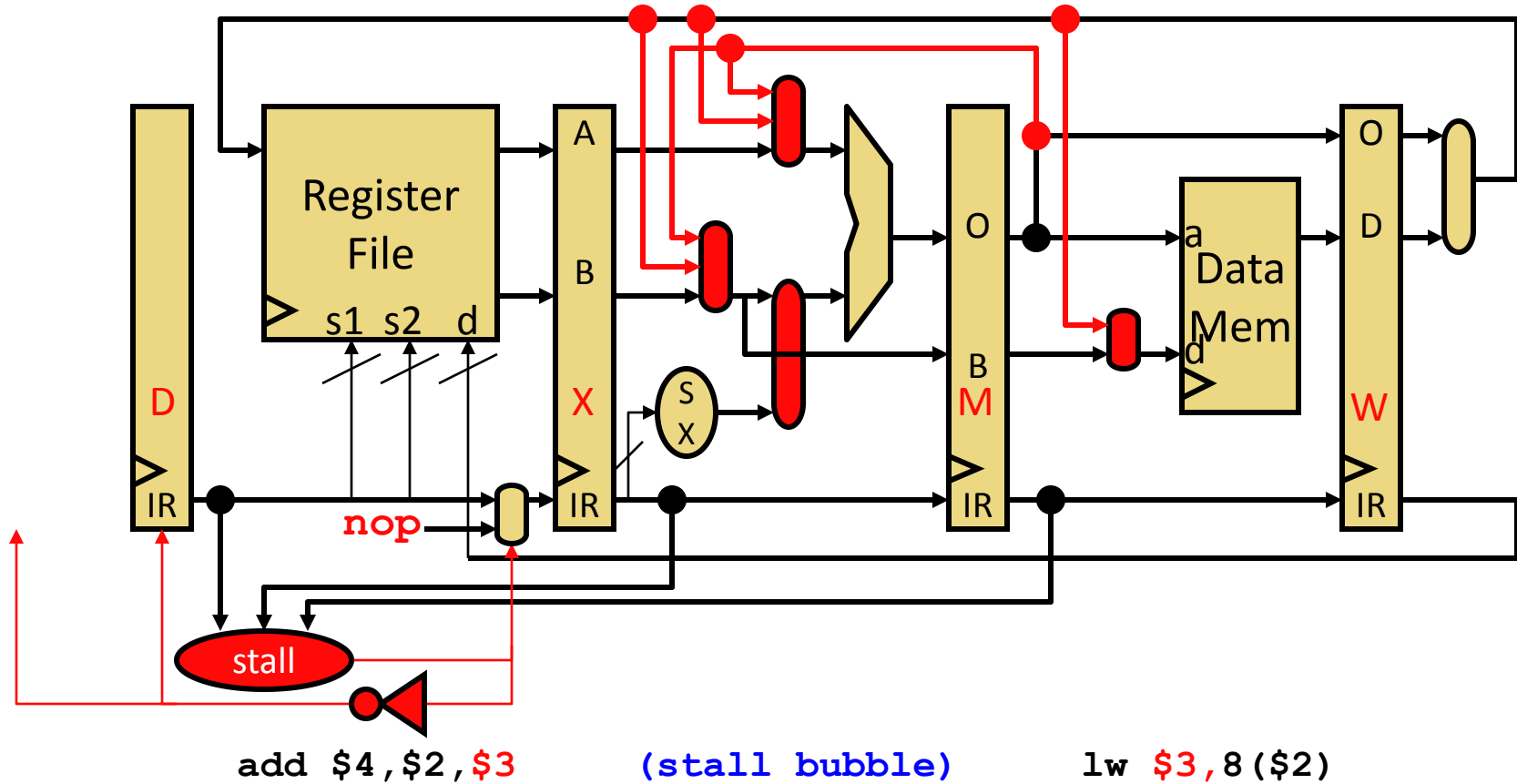
- 在D阶段阻止add前进
 - ✓ 向X.IR写入nop
 - ✓ 保持D阶段状态和PC状态不变
- 下一个周期再次检测

硬件方法（解决Load-To-Use相关性）



Stall = (X.IR.Operation == LOAD) &&
 ((D.IR.RegSrc1 == X.IR.RegDest) ||
 ((D.IR.RegSrc2 == X.IR.RegDest) && (D.IR.Op != STORE))
)

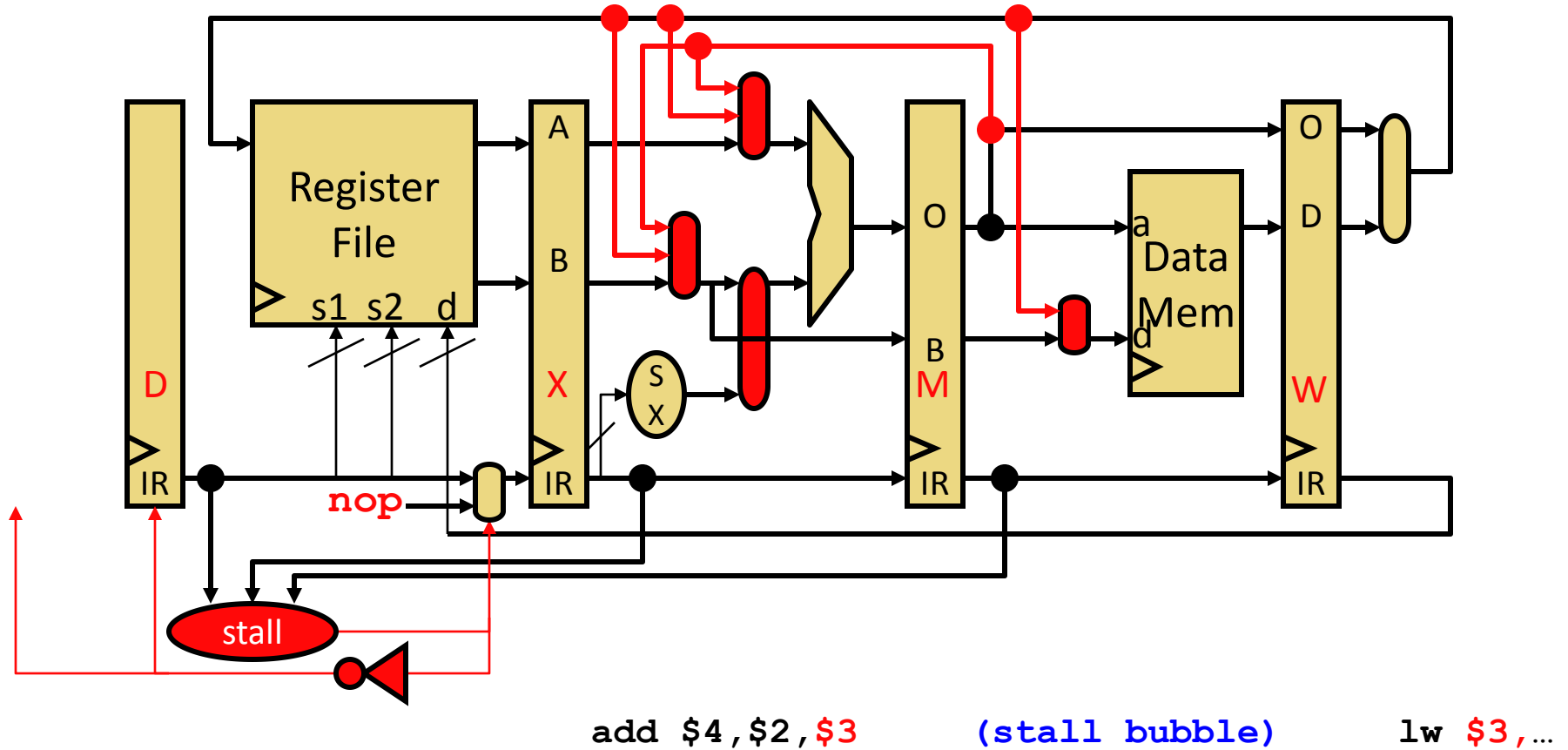
硬件方法（解决Load-To-Use相关性）



```

Stall = (X.IR.Operation == LOAD) &&
        ( (D.IR.RegSrc1 == X.IR.RegDest) ||
          ((D.IR.RegSrc2 == X.IR.RegDest) && (D.IR.Op != STORE))
        )
    
```

硬件方法（解决Load-To-Use相关性）



```

Stall = (X.IR.Operation == LOAD) &&
        ( (D.IR.RegSrc1 == X.IR.RegDest) ||
          ((D.IR.RegSrc2 == X.IR.RegDest) && (D.IR.Op != STORE))
        )
    
```


流水线深度 (Pipeline Depth)

□ **无流水线**: 指令执行需要 **N** 个阶段, 每个阶段需要 t_n 时间

- L_1 (单条指令执行时间) = $\sum t_n$
- T (吞吐量) = $1/L_1$
- L_M (M条指令执行时间, $M \gg 1$) = $M * L_1$

□ **N级流水线** :

- $L_{1+p} = L_1$ (单条指令的执行时间不变)
- $T_{+p} = 1/\max(t_n) \leq \mathbf{N}/L_1$ (理想情况下吞吐量变为N倍)
 - 如果每个阶段的 t_n 都相同, 正好变为N倍
- $L_{M+p} = M * \max(t_n) \geq M * L_1 / \mathbf{N}$ (M条指令执行时间)
- S_{+p} (加速比) = $[M * L_1 / (\geq M * L_1 / N)] = \leq \mathbf{N}$

问题: N越大, 加速比越高?

□ 假设: 每个流水段增加的额外延迟为O

- 寄存器开销: 读写寄存器需要时间

□ 带有额外开销的N级流水线:

- 假设 $\max(t_n) = L_1/N$
- $L_{1+P+O} = L_1 + N*O$
- $T_{+P+O} = 1/(L_1/N + O) = 1/(1/T + O) \leq T, \leq 1/O$
- $L_{M+P+O} = M*L_1/N + M*O = L_{M+P} + M*O$
- $S_{+P+O} = [M*L_1 / (M*L_1/N + M*O)] = \leq N = S_{+P}, \leq L_1/O$

□ **O** 限制了加速比的上限，N再大也没用

流水线深度的影响

□ 增加流水线深度

- 将指令执行过程划分成更多阶段
- + 增加时钟频率 (降低时钟周期)
 - 寄存器开销 & 流水段不均衡划分导致非线性增加
 - 流水线深度翻倍换不来时钟频率加倍
- 增加 CPI (降低 IPC)
 - 更多 “hazards”, 更高的分支惩罚 (branch penalty)
 - 内存访问延迟相对增加 (绝对时间没变, 周期数增加了)

结论: 深度大于某个值时, 流水线越深性能越差

截至目前，流水线：

- Scalar pipeline
- In-order pipeline
- ~~➤ 指令无数据相关性~~
- 无分支指令
- 所有指令单个周期完成

